

✓ Particle with a Finite Potential

- **Author:** Boran Aybak Kilic
- **Date:** 05.02.2026
- **Time spent on this assignment:** 5.5 hours

```
1 import numpy as np
2 import scipy
3 import matplotlib.pyplot as plt
4 import math
5 from matplotlib.animation import FuncAnimation
6 from IPython.display import HTML
7 import matplotlib.animation as animation
8 from IPython.display import HTML
9 def resetMe(keepList=[]):
10     ll=%who_ls
11     keepList=keepList+['FuncAnimation','HTML','resetMe','scipy']
12     for iiii in keepList:
13         if iiii in ll:
14             ll.remove(iiii)
15     for iiii in ll:
16         jjjj="^"+iiii+"$"
17         %reset_selective -f {jjjj}
18     ll=%who_ls
19     plt.rcParams.update({"font.size": 14})
20     return
21 resetMe()
22 import datetime;datetime.datetime.now()
```

```
datetime.datetime(2026, 5, 5, 16, 5, 22, 1940)
```

In this assignment, we will consistently use grids and observables that we will set up in this way. This is very similar to how you set things for the particle in the box assignment. We've also updated your animation code to include a skip parameter which will allow you to only show every skip'th frame. This will make your animation generate much faster (if you choose a big skip). It can't be called like

```
fig, ax = plt.subplots()
skip=10
max_value = np.max([np.max(np.abs(array)**2) for array in arrays[1:]])
animation = FuncAnimation(fig, update, frames=len(arrays)//skip, interval=1000)
display(HTML(animation.to_jshtml()))
plt.close()
```

```

1 def SetupGrid(L,delta_x):
2     n=int(round(L/delta_x))+1
3     xs=np.linspace(-L/2,L/2,n,endpoint=True)
4     return xs
5
6 def SetupObservables(xs,delta_x):
7     X=np.diag(xs)
8     P=(np.diag([-1.j/(2*delta_x) for i in range(len(xs)-1)],k=1
9
10     P2=np.zeros_like(P)
11     for i in range(len(xs)):
12         P2[i,i]=2.0/delta_x**2
13         if i+1<len(xs):
14             P2[i,i+1]=-1.0/delta_x**2
15             P2[i+1,i]=-1.0/delta_x**2
16     return X,P,P2
17
18 def update(frame, skip, xs, positions=None, potential=[], max_v
19     plt.cla() # Clear the current plot
20     scale=max_value/energy
21     plt.axhline(energy*scale, color='red', linestyle='--')
22     plt.plot(xs,potential*scale)
23     if type(positions)==np.ndarray:
24         plt.axvline(positions[::skip][frame], color='green', li
25     elif positions!=None:
26         print("ERROR")
27         return
28     plt.plot(xs, np.abs(arrays[::skip][frame])**2) # Plot the
29     plt.ylim(0, 1.5*max_value) # Set the y-axis limit
30     plt.xlabel('x')
31     plt.ylabel('Value')
32     plt.title(f'Frame {skip*frame+1}/{len(arrays)}') # Display
33

```

✓ Exercise 1. Particle in a Finite Well

In the previous assignment we worked on a particle in a box with infinite walls. In this assignment, we are going to work instead with a potential well again looking at properties such as eigenstates and time dynamics.

✓ a. Setting up the Hamiltonian

Consider a particle in a box of length 40 spanning

- $-10 \leq x \leq 10$ at $V = 0$
- $-20 \leq x \leq -10$ at $V = 0.5$
- $10 \leq x \leq 20$ at $V = 0.5$

Generate a discretize `xs` with the same $\delta x = 0.01$ that you used previously - i.e. `xs=SetupGrid(L,delta_x)` Also generate a numpy array `potential` which has the correct potential at each value of x . Verify this all looks correct by plotting V vs. x .

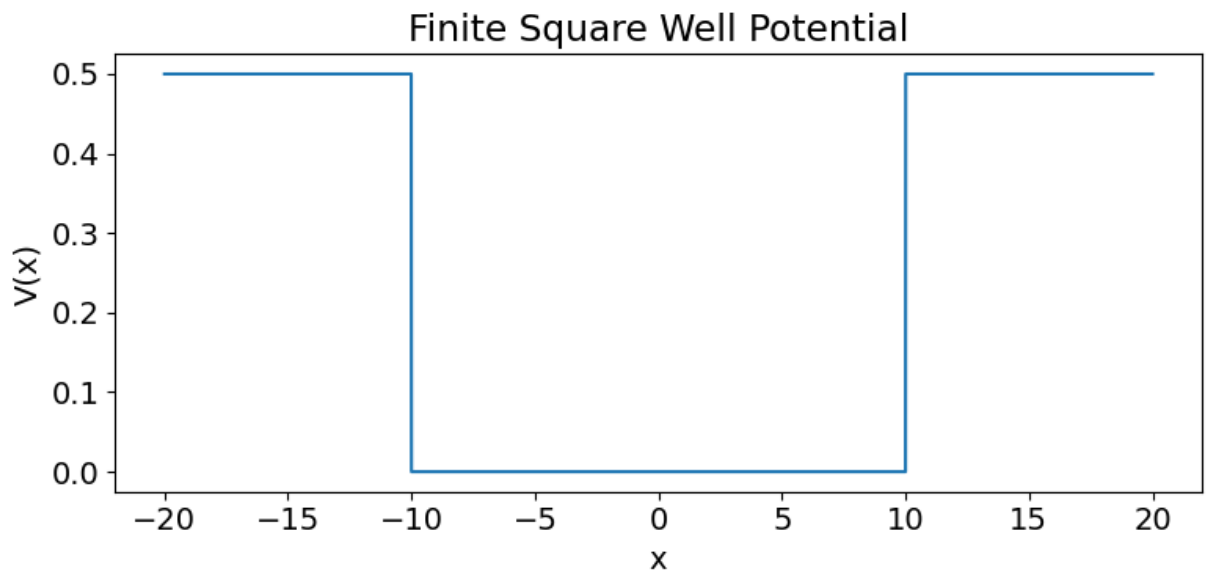
It will also be convenient here to setup your observables using

```
X,P,P2=SetupObservables(xs,delta_x)
```

although we won't use them till later

? Answer (start)

```
1 L = 40
2 delta_x = 0.01
3 xs = SetupGrid(L, delta_x)
4
5 potential = np.where(np.abs(xs) <= 10, 0.0, 0.5)
6
7 X, P, P2 = SetupObservables(xs, delta_x)
8
9 plt.figure(figsize=(8, 4))
10 plt.plot(xs, potential)
11 plt.xlabel('x')
12 plt.ylabel('V(x)')
13 plt.title('Finite Square Well Potential')
14 plt.tight_layout()
15 plt.show()
```



✓ Answer (end)

We know need to again set up the Hamiltonian

$$H \equiv \left(-\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} + V \right)$$

You already know how to turn the first term into a matrix. Because we are working in the x basis, the V term is also very simple: it is just a matrix with $V(x)$ (`np.diag(V)`) on the diagonal. To get the matrix for the total Hamiltonian, we just add these two matrices.

Set up this Hamiltonian, and then get the eigenvalues and eigenvectors.

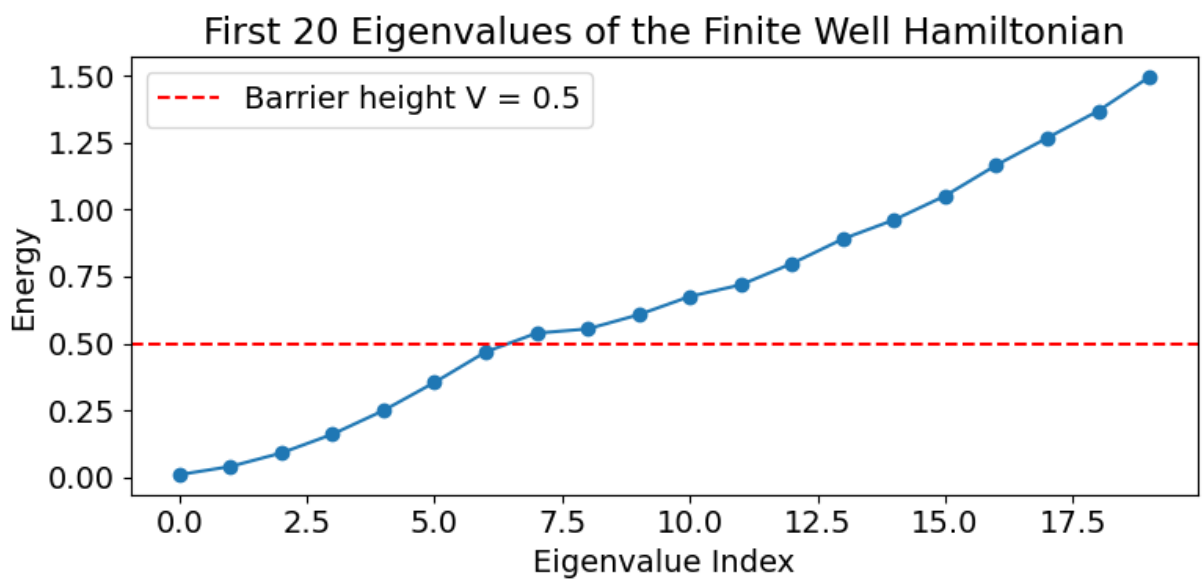
Plot the first 20 eigenvalues. You should notice that there is two qualitatively different behavior happening that switches over when the eigenvalue energy gets greater then 0.5 (where our potential wall is.)

? Answer (start)

```

1 H = 0.5 * P2 + np.diag(potential)
2 e, v = np.linalg.eigh(H)
3
4 plt.figure(figsize=(8, 4))
5 plt.plot(range(20), e[:20], 'o-', markersize=6)
6 plt.axhline(0.5, color='red', linestyle='--', label='Barrier height')
7 plt.xlabel('Eigenvalue Index')
8 plt.ylabel('Energy')
9 plt.title('First 20 Eigenvalues of the Finite Well Hamiltonian')
10 plt.legend()
11 plt.tight_layout()
12 plt.show()

```



Eigenvalues below $E = 0.5$ are discrete bound states with increasing spacing, while those above $E = 0.5$ become closely-packed, transitioning to the quasi-continuum of an unbound (scattering) particle confined to the full box.

✓ Answer (end)

Now we would like to plot the energies in a slightly different way and plot the eigenvectors on top of them. Let's start by again plotting the potential as a function of distance.

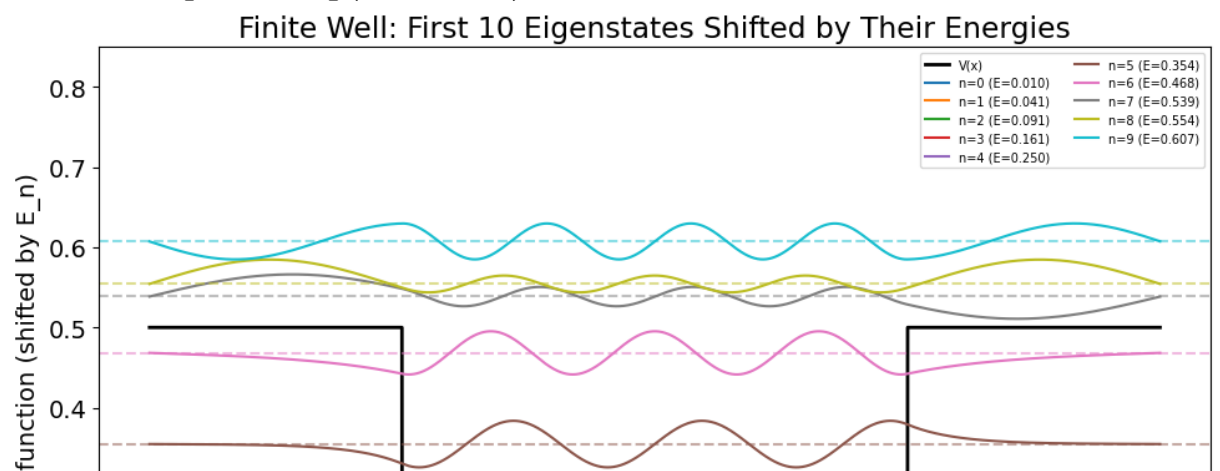
On top of this potential we are going to plot the lowest 10 energy levels. Plot them with `plt.axhline(the_ith_energy, linestyle='--', alpha=0.5)` You should notice that some of the energies are less than the potential and some of them more than the potential.

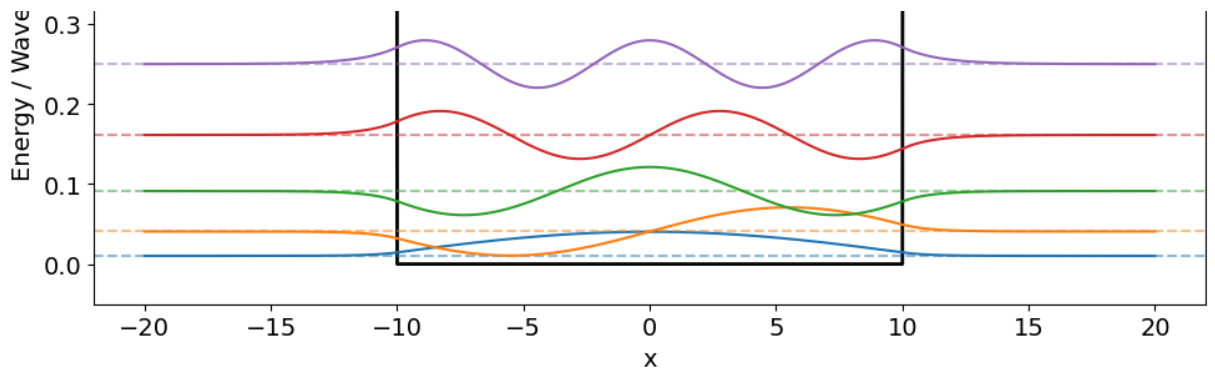
Now on top of this, we want to plot the lowest 10 eigenstates. To make them easier to understand, let's shift each eigenstate vertically by adding `e[i]` to the *i*'th eigenstate - i.e. `plt.plot(v[:,i]+e[i])`. Notice that if you do this, you have each eigenstate whose origin is centered on their energy.

? Answer (start)

```
1 plt.figure(figsize=(10, 7))
2 plt.plot(xs, potential, 'k-', linewidth=2, label='V(x)')
3 for i in range(10):
4     plt.axhline(e[i], linestyle='--', alpha=0.5, color=f'C{i}')
5     plt.plot(xs, v[:, i] + e[i], color=f'C{i}', label=f'n={i} (
6 plt.ylim(-0.05, 0.85)
7 plt.xlabel('x')
8 plt.ylabel('Energy / Wavefunction (shifted by E_n)')
9 plt.title('Finite Well: First 10 Eigenstates Shifted by Their E
10 plt.legend(loc='upper right', fontsize=7, ncol=2)
11 plt.tight_layout()
12 plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/matplotlib/cbook.py:1709: Co
return math.isfinite(val)
/usr/local/lib/python3.12/dist-packages/matplotlib/cbook.py:1345: Co
return np.asarray(x, float)
```





✓ Answer (end)

✓ b. Eigenstates in the forbidden region

We now want to look at the eigenstates in the forbidden region. To begin with, let's just qualitatively notice the difference between the eigenstates which are lower in energy than the finite wall and the ones that are higher in energy than the finite wall.

Q: As the wave-function goes to $x > 10$, do the eigenstates approach zero

- for the eigenstates whose energy is greater than 0.5
- for the eigenstates whose energy is less than 0.5

When the energy is larger than the potential, notice that the larger the value of $(E - V)$ (at a given x), the shorter the wavelength of the wave.

When the energy is smaller than the potential (i.e. in the forbidden region), we know that the eigenstate should decay exponentially.

We have learned that when an eigenstate is in the forbidden region it should decay exponentially.

That means that if we look at any of the eigenstates with $e_i < 0.5$, we should find that they approach zero exponentially. It might look that way by eye, but we want to check this explicitly.

First, we need to learn how to extract the part of the wave-function where the energy is less than 0.5. To do this, we should define a mask - i.e. `mask=xs>10` to focus on the forbidden region.

Then we can plot the absolute value of the eigenstate (because the eigenstate can approach zero from below) against x s only in this region - i.e.

`plt.plot(xs[mask], v[:,n][mask])` for eigenstates $n \in \{0, 2, 4\}$. Label each of your plots by including as a keyword `label=n` and then add a `plt.legend()` at the end.

Then go ahead and plot it on a log-scale - `plt.yscale('log')`. If it looks linear on a log scale, this means that the wave-function is decaying exponentially. In practice, it may look like it diverges from that linear curve at large x . This is essentially because you are running into machine precision. Notice that the lower-energy eigenstates have larger slopes (i.e. decay faster); this is because they are deeper in the forbidden region.

We can actually quantify this (*this part is optional but interesting if you want to show that you get exactly the correct exponential decay*). Our expectation is that the wave-function goes as e^{-kx} where

$$k = \sqrt{\frac{2m(E - V_0)}{\hbar^2}}$$

This means that if we plot the slopes squared of these three lines versus $e_i - 0.5$ where e_i is the eigenstate energy, it should be a straight line with a slope of -2.

In order to get the slopes of the three lines, we can use python to fit them. Again let's define a mask that is simultaneously both in the forbidden region and not at large x where the machine precision starts to cause trouble:

`mask=np.logical_and(xs>10,xs<16)`

Now use `np.polyfit(x,y,1)[0]` to get the slope. Remember that `y` needs to be the log of the wave-function (appropriately masked).

Then plot the slope-squared versus the energy difference from 0.5 You should find

a straight line.

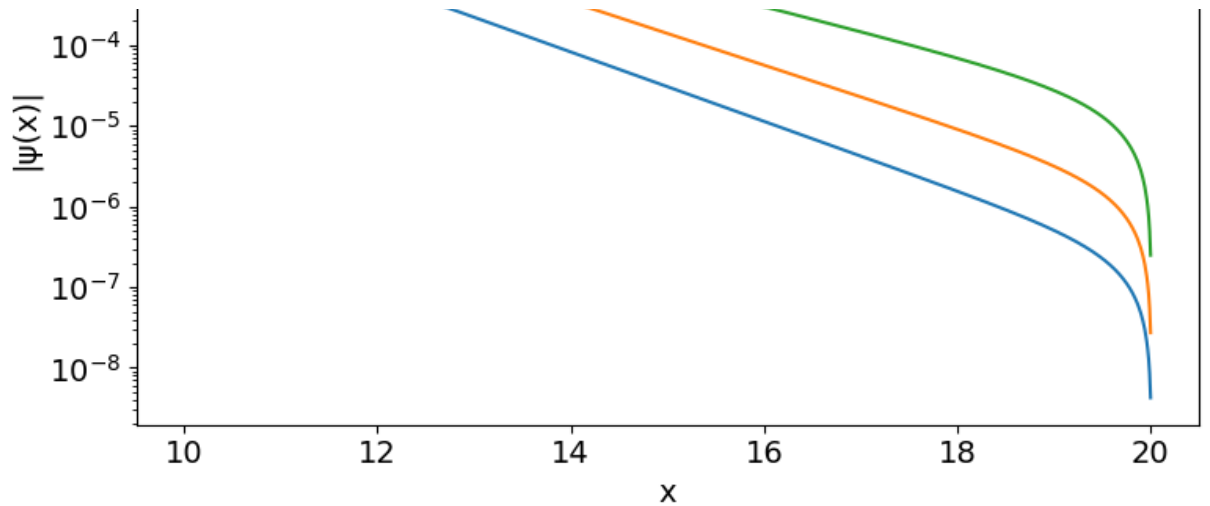
? Answer (start)

```

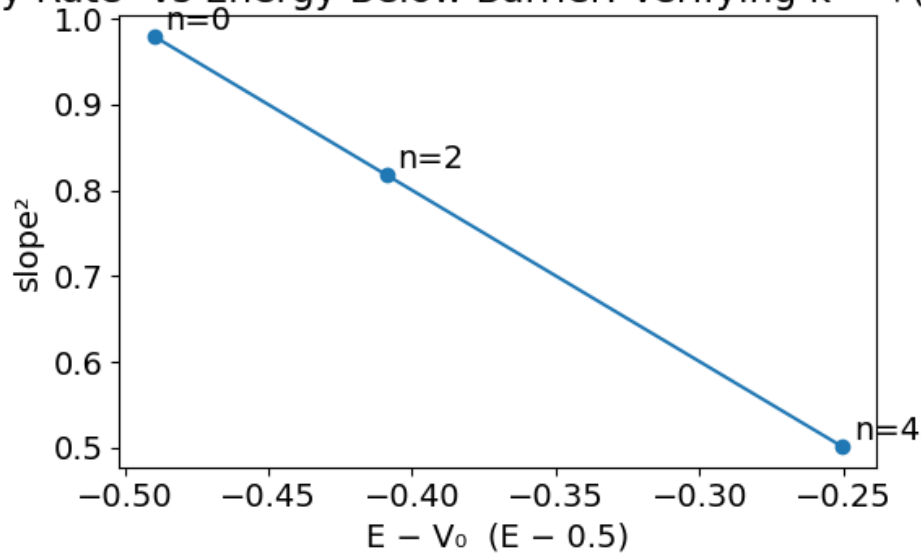
1 # Plot eigenstates in forbidden region (x > 10) on log scale
2 plt.figure(figsize=(8, 5))
3 mask = xs > 10
4 for n in [0, 2, 4]:
5     plt.plot(xs[mask], np.abs(v[:, n][mask]), label=f'n={n}', E=
6 plt.yscale('log')
7 plt.xlabel('x')
8 plt.ylabel('|ψ(x)|')
9 plt.title('Eigenstates in Forbidden Region (x > 10), Log Scale')
10 plt.legend()
11 plt.tight_layout()
12 plt.show()
13
14 # Fit exponential decay slopes in the well-behaved part of the
15 mask_fit = np.logical_and(xs > 10, xs < 16)
16 slopes = []
17 energies_fit = []
18 for n in [0, 2, 4]:
19     log_psi = np.log(np.abs(v[:, n][mask_fit]) + 1e-30)
20     slope = np.polyfit(xs[mask_fit], log_psi, 1)[0]
21     slopes.append(slope)
22     energies_fit.append(e[n])
23
24 slope_sq = [s**2 for s in slopes]
25 energy_diff = [en - 0.5 for en in energies_fit]
26
27 plt.figure(figsize=(6, 4))
28 plt.plot(energy_diff, slope_sq, 'o-')
29 for i, n in enumerate([0, 2, 4]):
30     plt.annotate(f'n={n}', (energy_diff[i], slope_sq[i]),
31                 textcoords='offset points', xytext=(5, 3))
32 plt.xlabel('E - V₀ (E - 0.5)')
33 plt.ylabel('slope²')
34 plt.title('Decay Rate² vs Energy Below Barrier: Verifying k = √')
35 plt.tight_layout()
36 plt.show()

```





Decay Rate² vs Energy Below Barrier: Verifying $k = \sqrt{2(V_0 - E)}$



The wavefunctions decay exponentially in the forbidden region (linear on a log scale), and the linear relationship $\text{slope}^2 \propto (V_0 - E)$ confirms the quantum mechanical prediction $k = \sqrt{2m(V_0 - E)/\hbar^2}$ with $m = \hbar = 1$.

✓ Answer (end)

✓ c. Dynamics

Like we did in the particle in the box, we are going to do some dynamics.

Because we are going to do some non-trivial time evolution in this assignment, it's worth encapsulating your time-evolution into two functions:

- `TimeEvolutionOperator(H,delta_t)` should take the Hamiltonian and your time step and return e^{-iHt} . It's important to note that for each Hamiltonian, you only need to compute this time-evolution operator once even if you are time-evolving many different wave-functions. If you are careful about this, you will save yourself significant time.
- `TimeEvolution(psi,M,steps)` which takes the initial wave-function `psi`, the time-evolution operator `M` and the number of steps and returns a list of arrays that include the snapshot of the wave-function at every time step.

ou will need to give `def update(frame, skip, xs, positions, potential, max_value, energy)` the relevant additional parameters.

`max_value` is the largest probability your wave-function gets (which we've already been calculating for animation) and `energy` is the energy of our initial wave-function, which you can compute (recall the energy doesn't change as a function of time). You also need to send it the position array which you can compute using your `X` from `SetupObservables` (outside the function) - e.g.

```
X,P,P2=SetupObservables(xs,delta_x)
```

We will start with the wave-function (same superposition of eigenstates as our particle-in-the-box but somewhat different eigenstates so a somewhat different wave-function)

$$|\Psi\rangle = \sqrt{0.1}|v_0\rangle + \sqrt{0.25}|v_1\rangle + i\sqrt{0.65}|v_2\rangle$$

Go ahead and animate this wave-function as a function of time again for a time-step $\delta t = 0.5$ and out to time $T = 200$.

Seperately and, in addition, plot the snapshots at $T = 0, 100, 200$ in one plot.

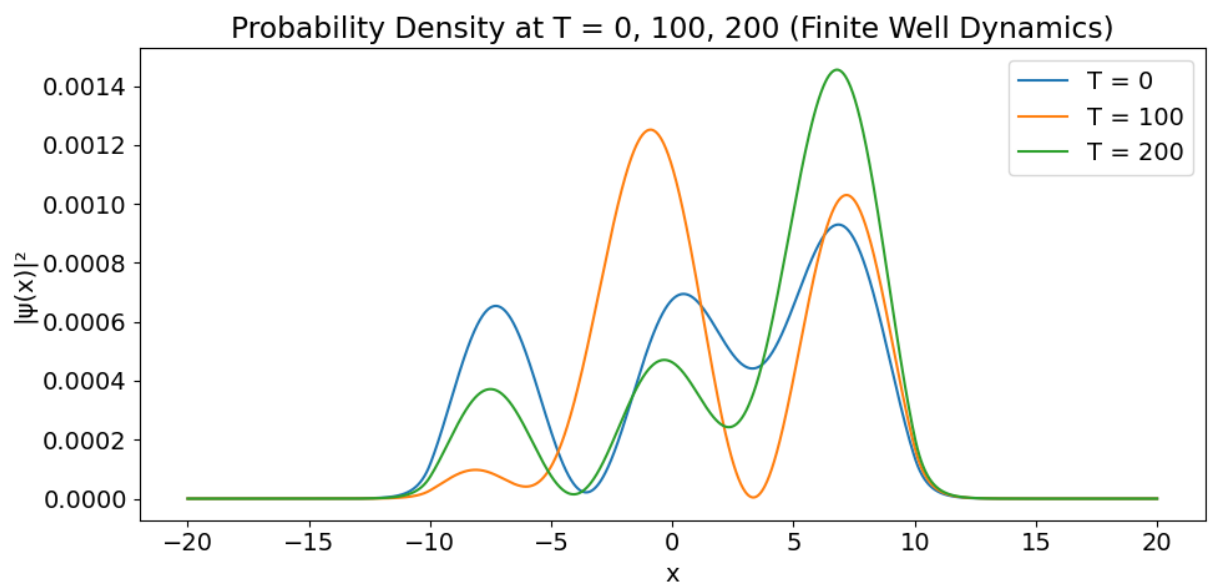
? Answer (start)

```
1 def TimeEvolutionOperator(H_op, delta_t):
2     return scipy.linalg.expm(-1.j * delta_t * H_op)
3
4 def TimeEvolution(psi_te, M_te, steps_te):
5     result = [psi_te.copy()]
6     for _ in range(steps_te):
7         psi_te = M_te @ psi_te
8         result.append(psi_te.copy())
9     return result
```

```

1 psi0 = np.sqrt(0.1) * v[:, 0] + np.sqrt(0.25) * v[:, 1] + 1.j *
2
3 delta_t = 0.5
4 T_total = 200
5 steps = int(T_total / delta_t)
6
7 M = TimeEvolutionOperator(H, delta_t)
8 arrays = TimeEvolution(psi0, M, steps)
9
10 # Snapshots at T = 0, 100, 200
11 plt.figure(figsize=(10, 5))
12 for t_snap in [0, 100, 200]:
13     frame_idx = int(t_snap / delta_t)
14     plt.plot(xs, np.abs(arrays[frame_idx])**2, label=f'T = {t_s
15 plt.xlabel('x')
16 plt.ylabel('|ψ(x)|²')
17 plt.title('Probability Density at T = 0, 100, 200 (Finite Well
18 plt.legend()
19 plt.tight_layout()
20 plt.show()

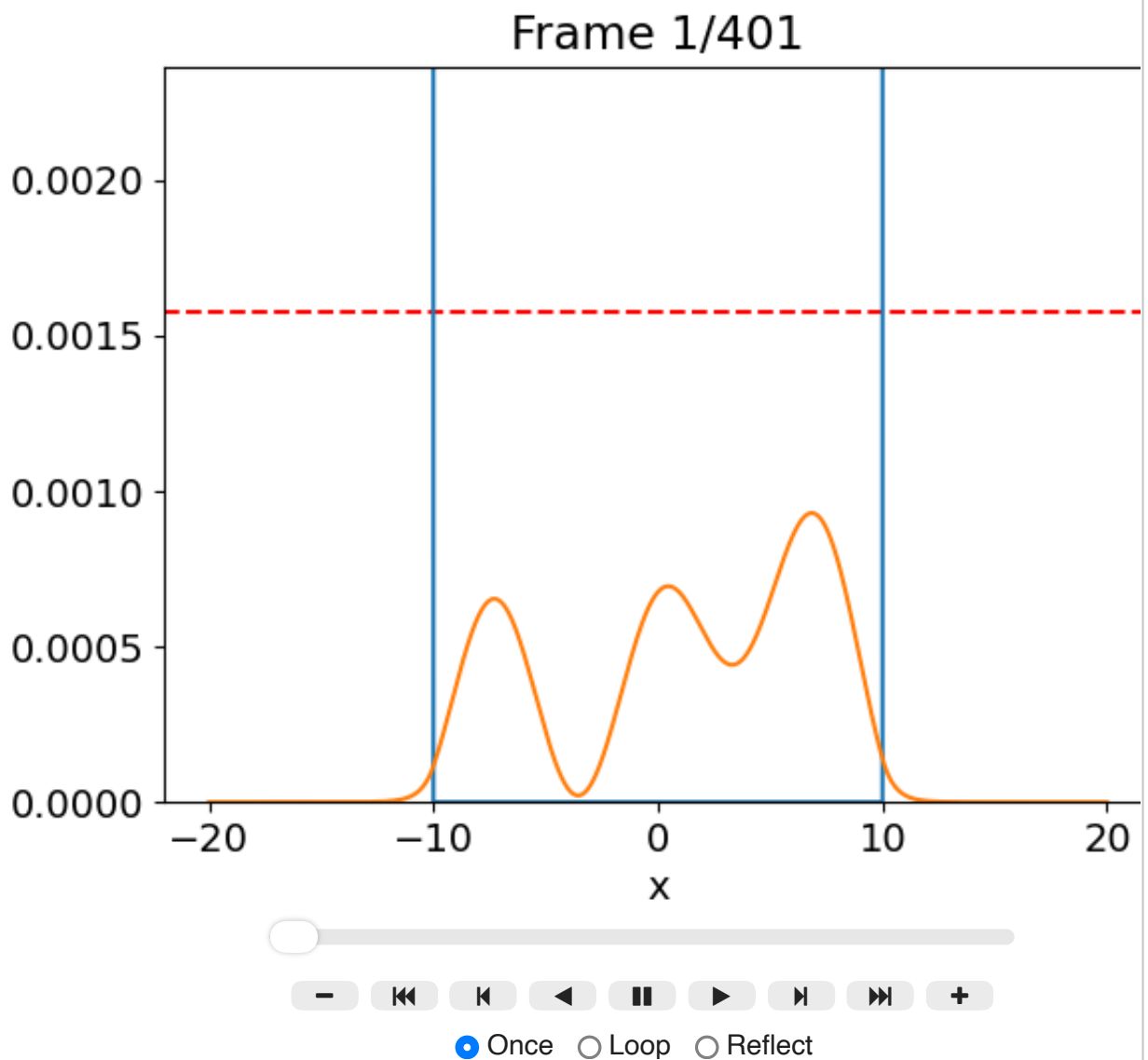
```



```

1 # Compute energy and max probability for animation scaling
2 energy_psi0 = np.real(psi0.conj() @ H @ psi0)
3 max_value = np.max([np.max(np.abs(arr)**2) for arr in arrays[1:
4
5 fig, ax = plt.subplots()
6 skip = 10
7 anim = FuncAnimation(fig, update, frames=len(arrays) // skip, i
8                      fargs=(skip, xs, None, potential, max_valu
9 display(HTML(anim.to_jshtml()))
10 plt.close()

```



✓ Answer (end)

If you go back and look at your animation from the particle-in-a-box, it actually should look very similar. The main different here is that the particle has leaked out slightly of the hard box (-10 to 10) and so you see some weight out there.

Let's also go ahead and plot some observables. Like you did in the particle-in-a-box go ahead and plot

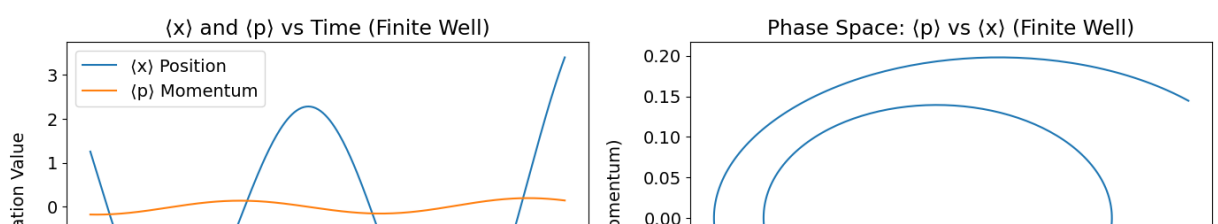
- the momentum and position as a function of time (on the same subplot)
- the phase plot of momentum vs position
- the energy as a function of time (making sure your scale is sane)

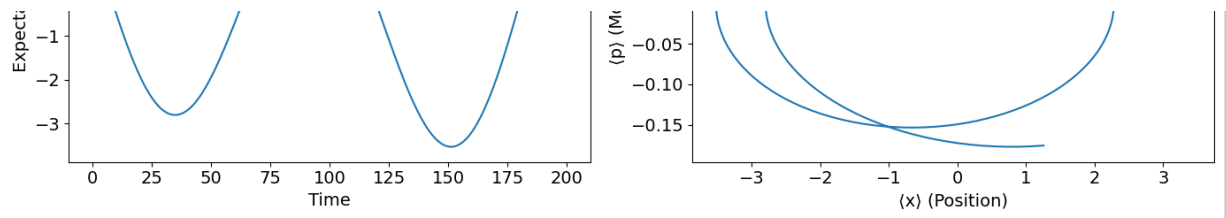
? Answer (start)

```

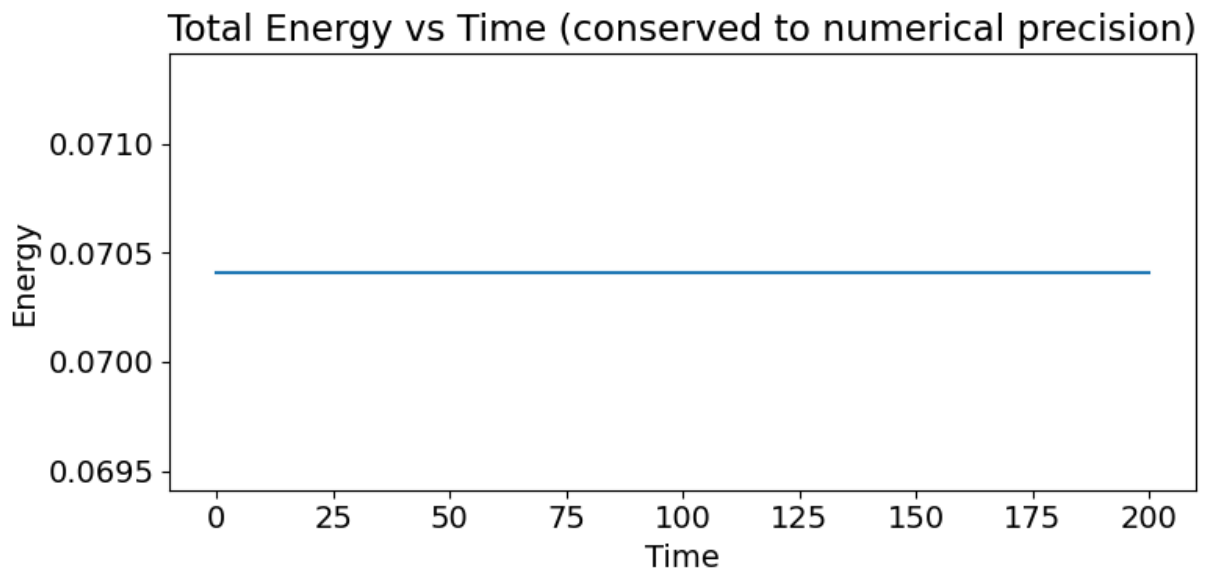
1 # Compute position, momentum, and energy as a function of time
2 positions_t = []
3 momenta_t = []
4 energies_t = []
5 times_t = np.arange(len(arrays)) * delta_t
6
7 for psi in arrays:
8     positions_t.append(np.real(psi.conj() @ X @ psi))
9     momenta_t.append(np.real(psi.conj() @ P @ psi))
10    energies_t.append(np.real(psi.conj() @ H @ psi))
11
12 # Position and momentum vs time on same subplot, plus phase spa
13 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
14 axes[0].plot(times_t, positions_t, label='⟨x⟩ Position')
15 axes[0].plot(times_t, momenta_t, label='⟨p⟩ Momentum')
16 axes[0].set_xlabel('Time')
17 axes[0].set_ylabel('Expectation Value')
18 axes[0].set_title('⟨x⟩ and ⟨p⟩ vs Time (Finite Well)')
19 axes[0].legend()
20
21 axes[1].plot(positions_t, momenta_t)
22 axes[1].set_xlabel('⟨x⟩ (Position)')
23 axes[1].set_ylabel('⟨p⟩ (Momentum)')
24 axes[1].set_title('Phase Space: ⟨p⟩ vs ⟨x⟩ (Finite Well)')
25 plt.tight_layout()
26 plt.show()

```





```
1 plt.figure(figsize=(8, 4))
2 plt.plot(times_t, energies_t)
3 e_mean = np.mean(energies_t)
4 plt.ylim(e_mean - 0.001, e_mean + 0.001)
5 plt.xlabel('Time')
6 plt.ylabel('Energy')
7 plt.title('Total Energy vs Time (conserved to numerical precision)')
8 plt.tight_layout()
9 plt.show()
10
```



Energy is conserved to numerical precision throughout the dynamics, while $\langle x \rangle$ and $\langle p \rangle$ oscillate quasi-periodically and trace a closed ellipse in phase space, reflecting quasi-harmonic confinement inside the finite well.

✓ Answer (end)

Again notice the marked similarity to a particle in a box with infinite walls.

✓ d. Dynamics above the potential barrier

In this problem, let us consider a superposition of eigenstates that is above the potential barrier. For example, let's take the superposition of eigenstates 9 and 10 (both above the potential) as

$$\Psi = \sqrt{0.35}|\nu_9\rangle + i\sqrt{0.65}|\nu_{10}\rangle$$

Produce the same set of animations and observable plots for this wave-function.

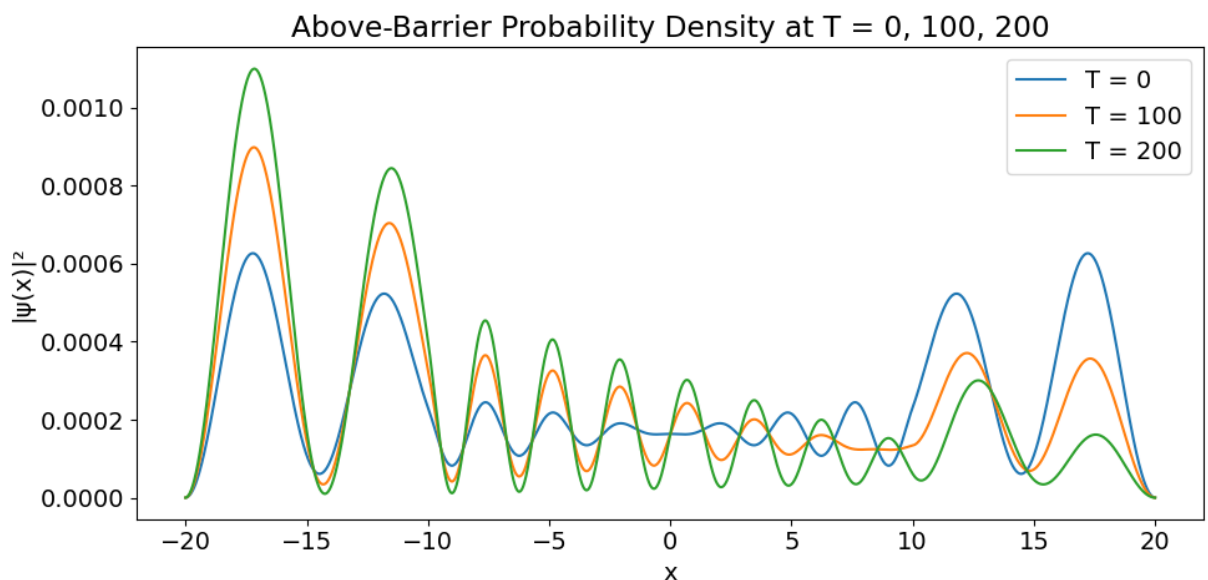
Notice in this case that significant chunks of the wave-function are in the forbidden region.

? Answer (start)

```

1 # Superposition of eigenstates 9 and 10 (both above the barrier
2 psi_d = np.sqrt(0.35) * v[:, 9] + 1.j * np.sqrt(0.65) * v[:, 10]
3
4 arrays_d = TimeEvolution(psi_d, M, steps)
5
6 # Snapshots at T = 0, 100, 200
7 plt.figure(figsize=(10, 5))
8 for t_snap in [0, 100, 200]:
9     frame_idx = int(t_snap / delta_t)
10    plt.plot(xs, np.abs(arrays_d[frame_idx])**2, label=f'T = {t_snap}')
11 plt.xlabel('x')
12 plt.ylabel('|ψ(x)|²')
13 plt.title('Above-Barrier Probability Density at T = 0, 100, 200')
14 plt.legend()
15 plt.tight_layout()
16 plt.show()

```



```

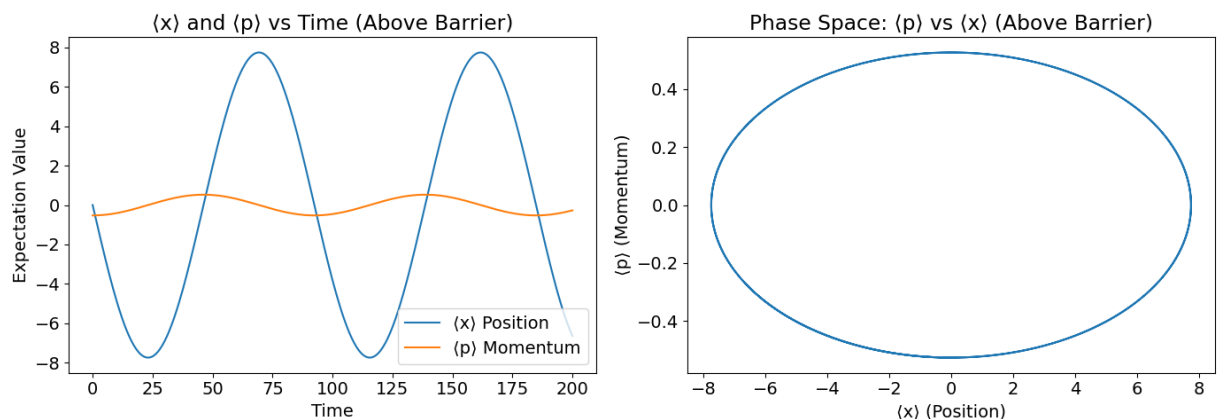
1 # Observables for above-barrier state
2 positions_d = []
3 momenta_d = []

```

```

4 energies_d = []
5
6 for psi in arrays_d:
7     positions_d.append(np.real(psi.conj() @ X @ psi))
8     momenta_d.append(np.real(psi.conj() @ P @ psi))
9     energies_d.append(np.real(psi.conj() @ H @ psi))
10
11 # Position and momentum vs time, plus phase space
12 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
13 axes[0].plot(times_t, positions_d, label='⟨x⟩ Position')
14 axes[0].plot(times_t, momenta_d, label='⟨p⟩ Momentum')
15 axes[0].set_xlabel('Time')
16 axes[0].set_ylabel('Expectation Value')
17 axes[0].set_title('⟨x⟩ and ⟨p⟩ vs Time (Above Barrier)')
18 axes[0].legend()
19
20 axes[1].plot(positions_d, momenta_d)
21 axes[1].set_xlabel('⟨x⟩ (Position)')
22 axes[1].set_ylabel('⟨p⟩ (Momentum)')
23 axes[1].set_title('Phase Space: ⟨p⟩ vs ⟨x⟩ (Above Barrier)')
24 plt.tight_layout()
25 plt.show()

```

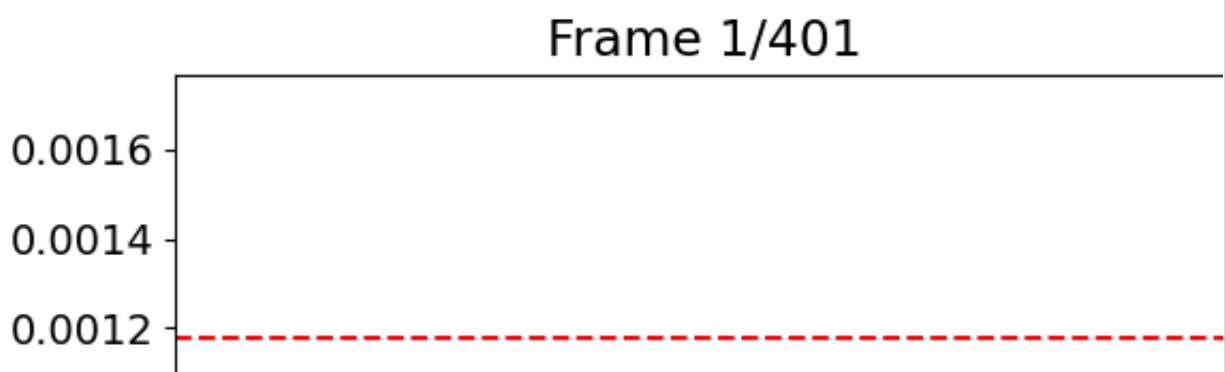
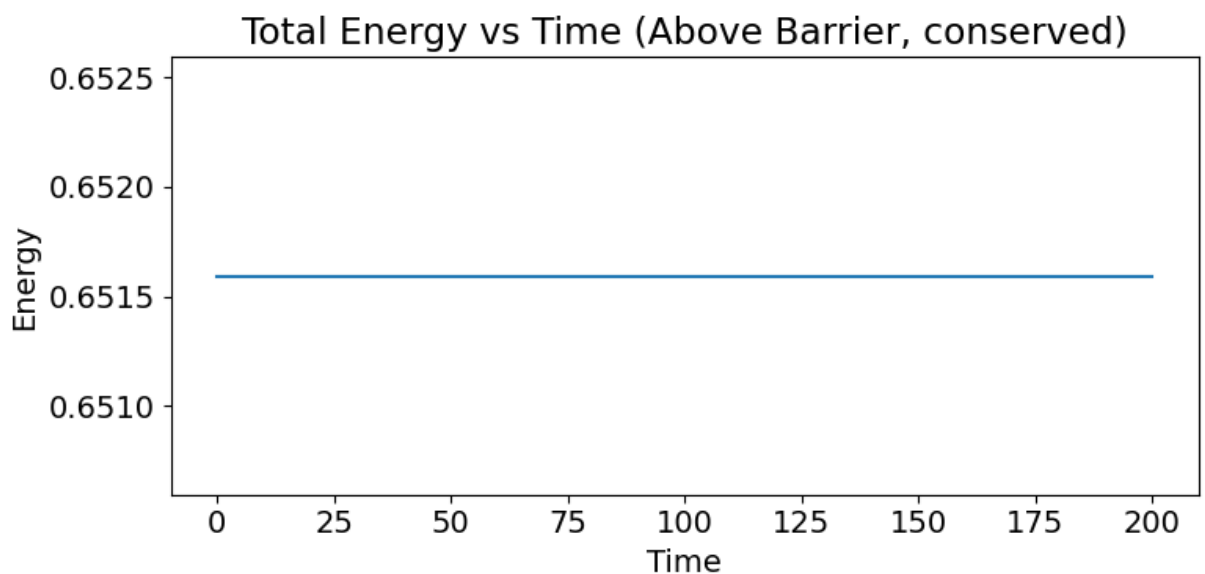


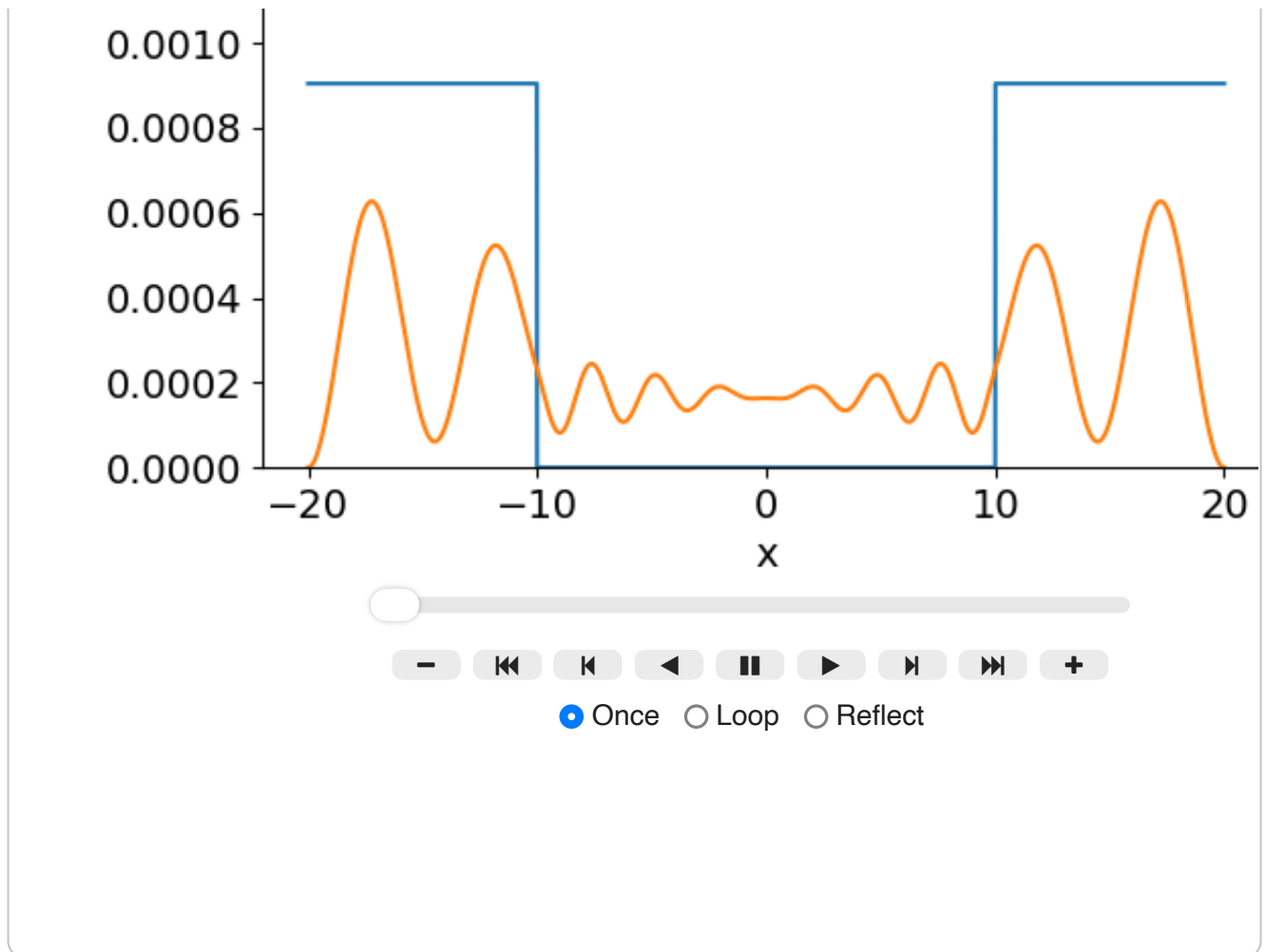
1 # Energy vs time

```

2 plt.figure(figsize=(8, 4))
3 plt.plot(times_t, energies_d)
4 e_d_mean = np.mean(energies_d)
5 plt.ylim(e_d_mean - 0.001, e_d_mean + 0.001)
6 plt.xlabel('Time')
7 plt.ylabel('Energy')
8 plt.title('Total Energy vs Time (Above Barrier, conserved)')
9 plt.tight_layout()
10 plt.show()
11
12 # Animation for above-barrier state
13 energy_d_val = np.real(psi_d.conj() @ H @ psi_d)
14 max_value_d = np.max([np.max(np.abs(arr)**2) for arr in arrays_
15 arrays = arrays_d
16
17 fig, ax = plt.subplots()
18 skip = 10
19 anim = FuncAnimation(fig, update, frames=len(arrays) // skip, i
20                      fargs=(skip, xs, None, potential, max_valu
21 display(HTML(anim.to_jshtml()))
22 plt.close()
23

```





Above the potential barrier, the wavefunction extends significantly into the classically forbidden region ($|x| > 10$) and the phase-space trajectory is more irregular, reflecting the richer spatial structure of unbound eigenstates.

✓ Answer (end)

- ✓ Exercise 2. Eigenstates of Finite Barriers
- ✓ a. Diagonalize

Now we will consider a potential with a finite barrier. Set the potential to be zero except for the range $-1 < x < 1$ where we will have the potential be $V_0 = 5$. Use $L = 20$

Start by plotting the potential.

Then plot the lowest two 5 eigenvalues and 2 eigenvectors. What you will find is that the two lowest eigenvectors are nearly degenerate - they have almost the same energy. Why is this? Can you tell from the actual eigenvectors that this would have been the case?

For this problem, it will run much faster if you use a more reasonable $\delta x = 0.1$

```
delta_x=0.1
xs=SetupGrid(L,delta_x)
```

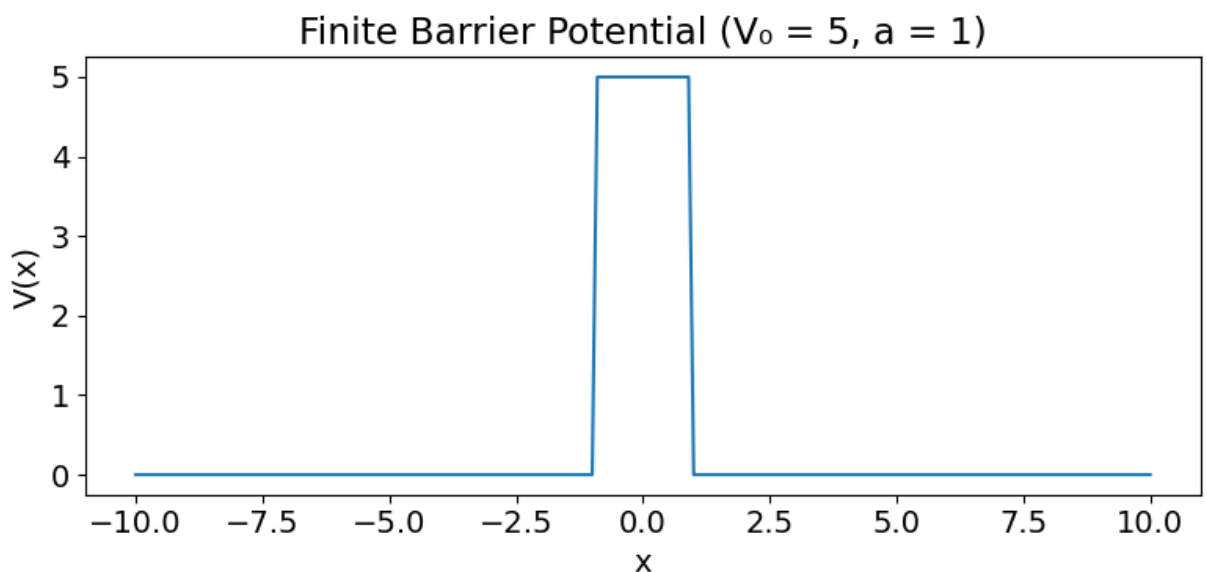
? Answer (start)

```
1 L = 20
2 delta_x = 0.1
3 xs = SetupGrid(L, delta_x)
4 X, P, P2 = SetupObservables(xs, delta_x)
5
6 # Finite barrier:  $V_0 = 5$  for  $-1 < x < 1$ , zero otherwise
7 potential_b = np.where(np.abs(xs) < 1, 5.0, 0.0)
8
9 plt.figure(figsize=(8, 4))
10 plt.plot(xs, potential_b)
11 plt.xlabel('x')
12 plt.ylabel('V(x)')
13 plt.title('Finite Barrier Potential ( $V_0 = 5$ ,  $a = 1$ )')
14 plt.tight_layout()
15 plt.show()
16
17 # Hamiltonian and eigenstates
18 H_b = 0.5 * P2 + np.diag(potential_b)
19 e_b, v_b = np.linalg.eigh(H_b)
20
21 # First 5 eigenvalues
22 plt.figure(figsize=(8, 4))
23 plt.plot(range(5), e_b[:5], 'o-', markersize=8)
24 plt.xlabel('Eigenvalue Index')
25 plt.ylabel('Energy')
26 plt.title('First 5 Eigenvalues of the Finite Barrier Hamiltonian')
```

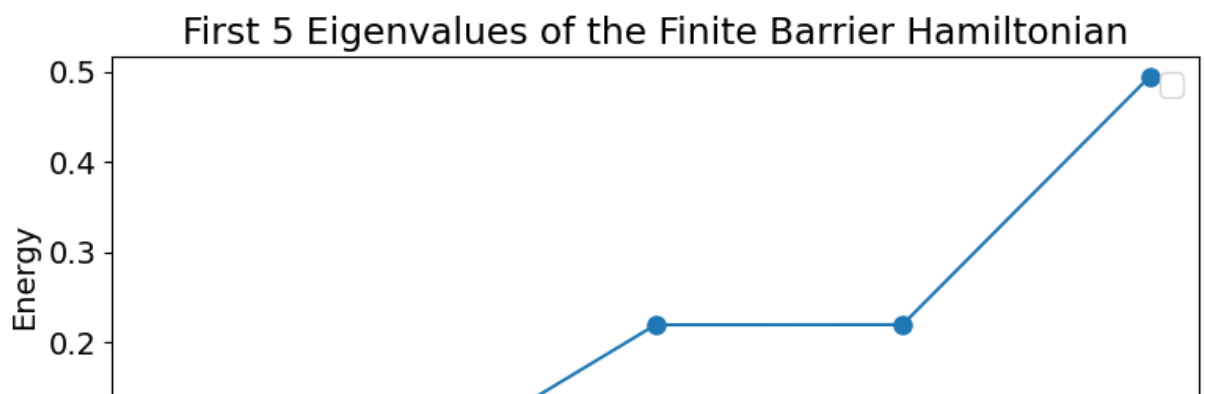
```

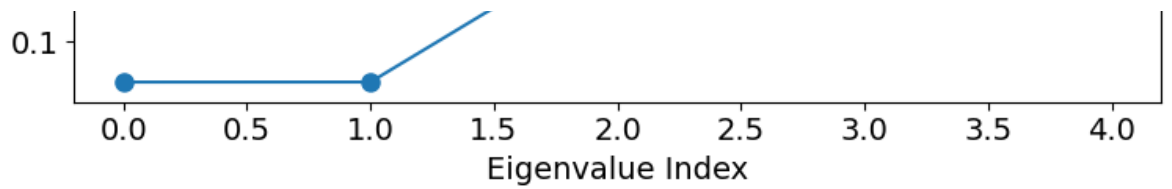
27 plt.legend()
28 plt.tight_layout()
29 plt.show()
30
31 # First 2 eigenvectors overlaid on potential
32 plt.figure(figsize=(10, 6))
33 for i in range(2):
34     plt.axhline(e_b[i], linestyle='--', alpha=0.5, color=f'C{i}')
35     if i == 0:
36         plt.plot(xs, v_b[:, i] + e_b[i], color=f'C{i}', linestyle='solid')
37     else:
38         plt.plot(xs, v_b[:, i] + e_b[i], color=f'C{i}', linestyle='solid')
39 plt.xlabel('x')
40 plt.ylabel('Wavefunction (shifted by E_n)')
41 plt.title('First 2 Eigenstates of the Finite Barrier (shifted by E_n)')
42 plt.legend(handles=[
43     plt.Line2D([0], [0], color='C0', linestyle='--', linewidth=4),
44     plt.Line2D([0], [0], color='C1', linestyle=':', linewidth=3)
45 ])
46 plt.tight_layout()
47 plt.show()

```



/tmp/ipykernel_3675/2221729966.py:27: UserWarning: No artists with label found, skipping callback for this artist.
plt.legend()

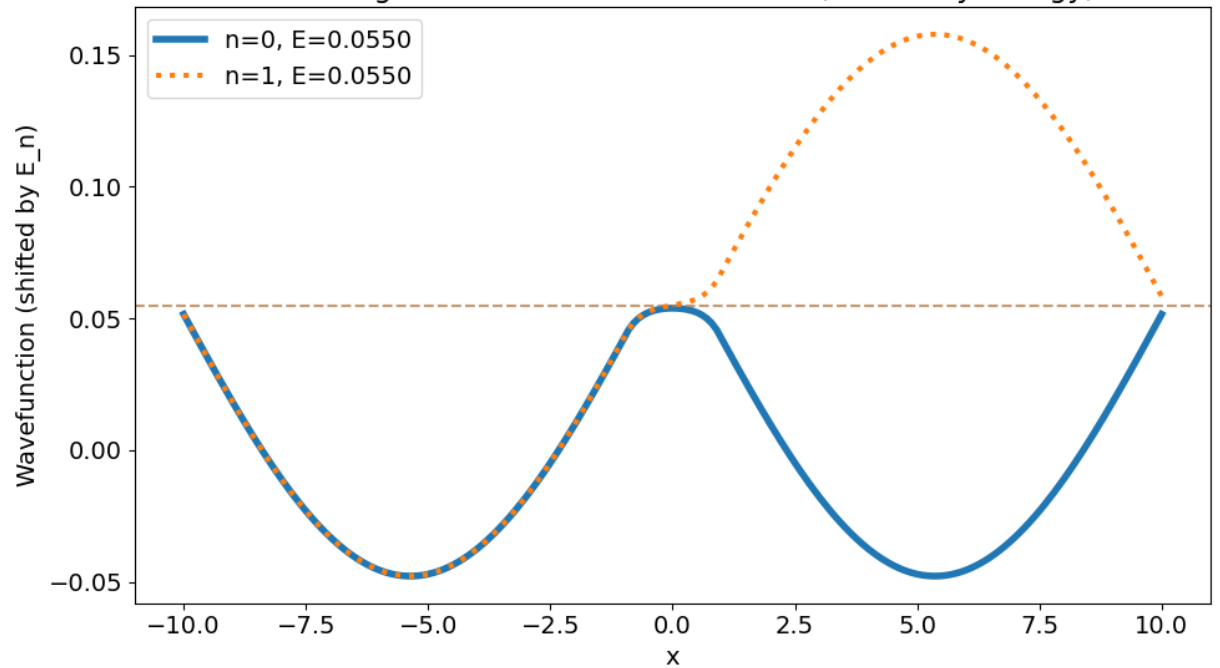




```
/usr/local/lib/python3.12/dist-packages/matplotlib/cbook.py:1709: Co
return math.isfinite(val)
```

```
/usr/local/lib/python3.12/dist-packages/matplotlib/cbook.py:1345: Co
return np.asarray(x, float)
```

First 2 Eigenstates of the Finite Barrier (shifted by energy)



The two lowest eigenstates are nearly degenerate because the central barrier divides the box into two weakly coupled wells; the symmetric and antisymmetric combinations of their ground states have nearly identical energies due to exponentially small tunneling through the barrier.

✓ Answer (end)

✓ Exercise 3. Faster Dynamics and Finite Barriers

So far we've done dynamics by evaluating $\exp[-i\Delta t H]$ for a finite Δt . This has the advantage of being the exact dynamics but when L is large or Δx is small this gets very expensive to calculate limiting our ability to do interesting calculations.

Here we are going to introduce an alternative approach. Consider the time-evolution operator

$$\exp \left[-i\Delta t \left(-\frac{\hat{P}^2}{2m} + \hat{V} \right) \right]^{T/\Delta t} |\Psi\rangle$$

When $\Delta t \rightarrow 0$, we can write this as

$$\left(\exp \left[-i\Delta t \left(-\frac{\hat{P}^2}{2m} \right) \right] \exp[-i\Delta t \hat{V}] \right)^{T/\Delta t} |\Psi\rangle$$

Instead of thinking of time-evolution as applying the exponential of the Hamiltonian, we can think of time evolution as alternatively applying the exponential of the potential and kinetic terms.

If our wave-function is a vector in the position basis (which it mainly has been), then applying the potential term (treating V as an array - earlier we called it `potential`) is very easy because multiplying by a diagonal matrix is the same as pointwise multiplying by a vector - i.e. all three of these lines below are equivalent:

```
scipy.linalg.expm(-1.j*dt*np.diag(V)) @ psi
np.diag(np.exp(-1.j*dt*V)) @ psi
np.exp(-1.j*dt*V) * psi
```

The first line is the sort of thing we've been doing all along (but just with the potential piece). The last line doesn't involve a matrix at all but just a vector. Let us define `M_half_x=np.exp(-1.j*dt/2*np.diag(V))`

Unfortunately, the kinetic piece is complicated if our wave-function is in the position basis. But.... if our wave-function is in the momentum basis, then it would be easy because in the momentum basis that matrix is diagonal so we can just pointwise multiply by the vector `M_p=np.exp(-1.j* dt* (ks**2)/(2*m))`.

But it's easy to move the wave-function back and forth between the position and momentum basis via an FFT so we can always be in the "good" basis when we have to apply the operator.

This leaves us with the following algorithm to apply a single time-step Δt starting with a wave-function `psi` in the position basis:

- (1) apply `M_half_x` to `psi`
- (2) apply `M_half_x` to `psi`
- (3) switch to the momentum basis `psi_k=np.fft.fft(psi)`
- (4) apply `M_p` to `psi_k`
- (5) switch to the real space basis `psi=np.fft.ifft(psi_k)`

We then need to do this over and over again for many time-steps (for technical reasons, it's actually more accurate to put (1) after (5)).

You are making a tiny error in Δt but this algorithm is much better than our previous approaches. You never need to build a big matrix diagonalize a Hamiltonian, etc. To implement this you will need the k-vectors

`ks=np.fft.fftfreq(len(xs), delta_x)*(2*np.pi)`

There is one additional wart. This will only work on wave-functions that go to zero at the edge of your box but since you can use such a large box this shouldn't be a problem.

✓ a. Fast Time evolution for the Finite Well

In this part, we will do time evolution again on the finite well but this time much more efficiently. The finite well parameters we will use are

- `dt=0.01`

- $\text{delta_x}=0.1$
- $L=40$ (for a width of 80)
- $T=1$ (this will be the time at which your Time Evolution runs - you will run it many times but each time saving a snapshot at each T)
- an initial wave-function $|\Psi\rangle = \sqrt{0.1}|\nu_0\rangle + \sqrt{0.25}|\nu_1\rangle + i\sqrt{0.65}|\nu_2\rangle$ generated from the finite-well potential with $V(x) = 0.5$ when $|x| > 10$ and $V(x) = 0$ otherwise. (to get Ψ , you will need to work with your Hamiltonian again which is slow - this is just for a test).

Write a function

```
def SetupWell():
    # do stuff here
    return psi_init,V,dt,T,xs,freqs
```

which returns the relevant initial wave-function, potential, time-step, time-per-snapshot, grid, and frequencies. recall that the frequencies that correspond to our fft are

- `freqs = np.fft.fftfreq(len(xs), delta_x)*(2*np.pi)`

Now write a function `def TimeEvolution(T,dt,V,psi,freqs)` which does the time-evolution returning the new `psi`.

Run your time-evolution out to time $400T$ like

```
psi,V,dt,T,xs,freqs=SetupWell()
arrays=[psi]
for step in range(0,400):
    psi=TimeEvolution(T, dt, V, psi, freqs)
    arrays.append(psi)
```

If you plot the probability density of the wave-function at time $40T$ you should see the wave-function in the potential barrier with the weight to the left.

```
plt.plot(xs,V/200)
plt.plot(xs,np.abs(arrays[40])**2)
```

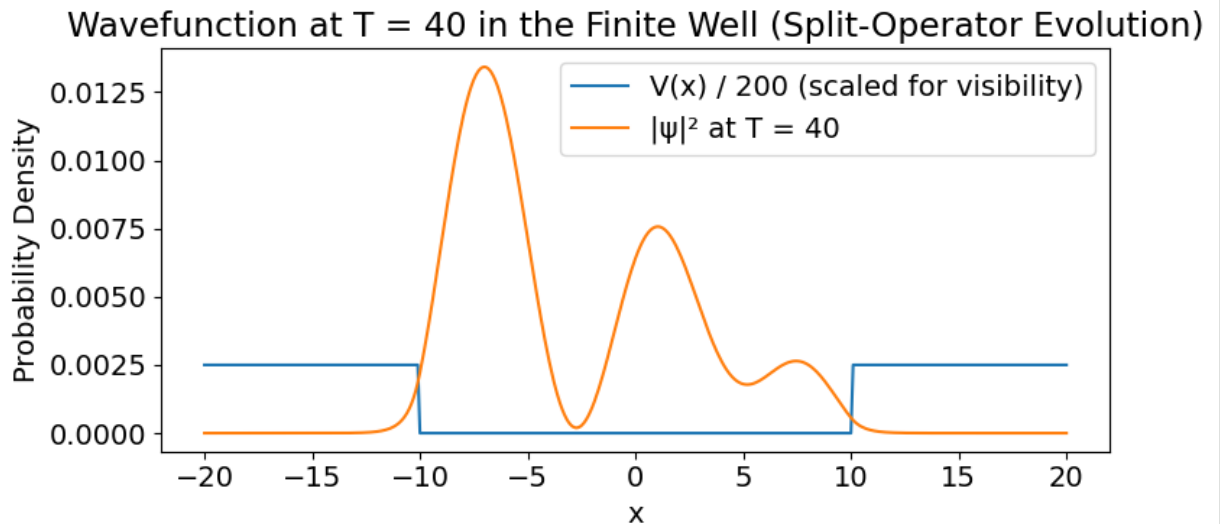
? Answer (start)

```

1 def SetupWell():
2     L_w = 40
3     delta_x_w = 0.1
4     dt_w = 0.01
5     T_w = 1.0
6     xs_w = SetupGrid(L_w, delta_x_w)
7     freqs_w = np.fft.fftfreq(len(xs_w), delta_x_w) * (2 * np.pi)
8     V_w = np.where(np.abs(xs_w) <= 10, 0.0, 0.5)
9
10    # Build Hamiltonian to get eigenstates for initial state cc
11    _, _, P2_w = SetupObservables(xs_w, delta_x_w)
12    H_w = 0.5 * P2_w + np.diag(V_w)
13    e_w, v_w = np.linalg.eigh(H_w)
14
15    psi_init = (np.sqrt(0.1) * v_w[:, 0] +
16                np.sqrt(0.25) * v_w[:, 1] +
17                1.j * np.sqrt(0.65) * v_w[:, 2])
18    return psi_init, V_w, dt_w, T_w, xs_w, freqs_w
19
20
21 def TimeEvolution(T_ev, dt_ev, V_ev, psi_ev, freqs_ev):
22     """Split-operator (Strang) time evolution via FFT."""
23     n_steps = int(round(T_ev / dt_ev))
24     M_half_x = np.exp(-1.j * dt_ev / 2.0 * V_ev)
25     M_p = np.exp(-1.j * dt_ev * freqs_ev**2 / 2.0)
26     for _ in range(n_steps):
27         psi_ev = M_half_x * psi_ev           # half potential st
28         psi_k = np.fft.fft(psi_ev)
29         psi_k = M_p * psi_k                 # full kinetic step
30         psi_ev = np.fft.ifft(psi_k)
31         psi_ev = M_half_x * psi_ev           # half potential st
32     return psi_ev
33
34
35 psi, V, dt, T, xs, freqs = SetupWell()
36 arrays = [psi]
37 for step in range(0, 400):
38     psi = TimeEvolution(T, dt, V, psi, freqs)
39     arrays.append(psi)
40
41 plt.figure(figsize=(8, 4))
42 plt.plot(xs, V / 200, label='V(x) / 200 (scaled for visibility)')
43 plt.plot(xs, np.abs(arrays[40])**2, label='|ψ|² at T = 40')
44 plt.xlabel('x')
45 plt.ylabel('Probability Density')
46 plt.title('Wavefunction at T = 40 in the Finite Well (Split-Ope
47 plt.legend()
48 plt.tight_layout()

```

```
49 plt.show()
```



✓ Answer (end)

✓ b. Plotting our time-evolution

Now we would like to plot some data.

We want to plot

- the probability density of the wave-function in real space at time $50T$
- the probability density of the wave-function in k-space at time $50T$
- the average position as a function of time (**you shouldn't need anything from SetupObservables for this such as the X or P**)
- the momentum as a function of time (you shouldn't need anything from SetupObservables for this)
- the total, kinetic, and potential energy as a function of time (on one plot) (you shouldn't need anything from SetupObservables for this)
- the potential

Write `GetEnergy(psis, freqs)` and `GetAverages(psis, xs, freqs)` which return these observables and plot them.

You can plot it however you want but I plotted it like this to get it into subplots

```
fig, ax = plt.subplots(2, 3)
fig.set_figwidth(22)
fig.set_figheight(12)

ax[0,0].plot(np.array(range(len(Es))*dt,Es,label='Total Energy')
ax[0,0].plot(np.array(range(len(PEs))*dt,PEs,label='Potential Energy')
ax[0,0].plot(np.array(range(len(KEs))*dt,KEs,label='Kinetic Energy')
ax[0,0].legend()
ax[0,0].set_xlabel('Time')
ax[0,0].set_ylabel('Energy')

ax[0,1].plot(np.array(range(len(positions))*dt,positions,label='Posit
ax[0,1].set_xlabel('Time')
ax[0,1].set_ylabel('Position')

ax[0,2].plot(np.array(range(len(momentums))*dt,momentums,label='Momen
ax[0,2].set_xlabel('Time')
ax[0,2].set_ylabel('Momentum')

ax[1,0].plot(xs,np.abs(arrays[50])**2)
ax[1,0].set_xlabel('x')
ax[1,0].set_ylabel('Probability Density')
psi_k=np.fft.fft(arrays[50])
ax[1,1].plot(freqs,np.abs(psi_k)**2,'.')
ax[1,1].set_xlabel('k')

ax[1,2].plot(xs,V)
ax[1,2].set_xlabel('x')
ax[1,2].set_ylabel('Potential Energy')
plt.tight_layout()
plt.show()
```

? Answer (start)

```
1 def GetEnergy(psis, freqs_g, V_g, xs_g):
2     """Return total, kinetic, and potential energy lists using F
3     dx = xs_g[1] - xs_g[0]
4     N = len(xs_g)
5     .. .. ..
```

```

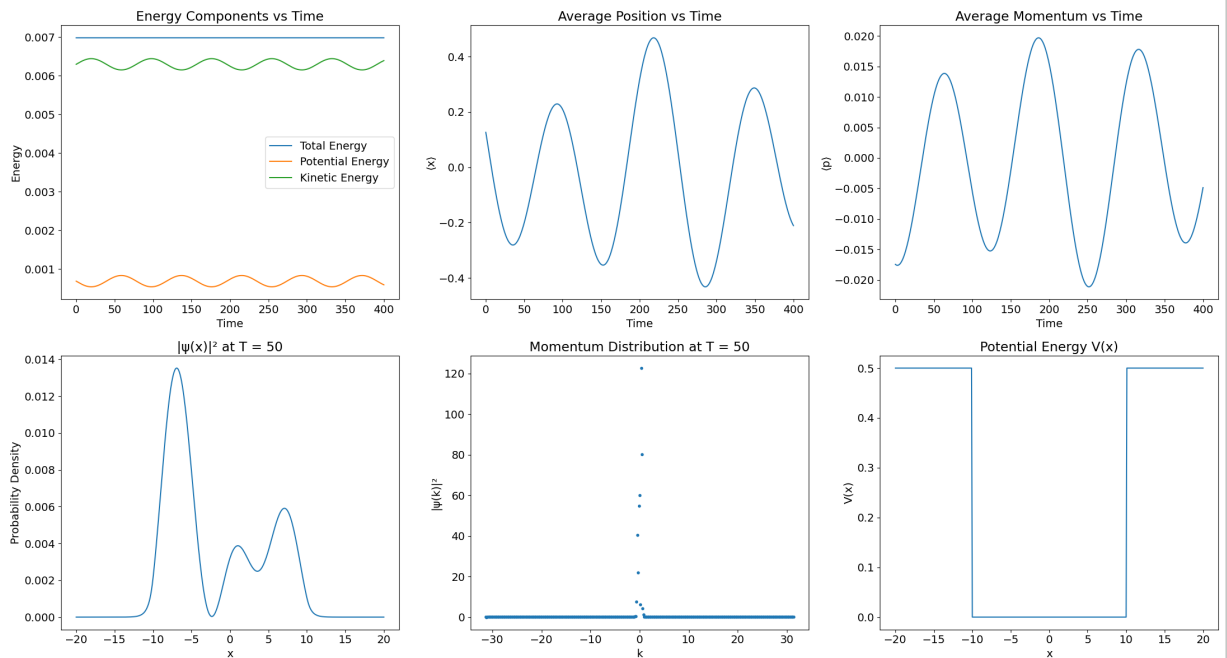
5     Es, KEs, PEs = [], [], []
6     for psi_g in psis:
7         PE = np.real(np.sum(np.abs(psi_g)**2 * V_g) * dx)
8         phi = np.fft.fft(psi_g)
9         KE = np.real(dx / N * np.sum(freqs_g**2 / 2.0 * np.abs(p
10        Es.append(PE + KE)
11        KEs.append(KE)
12        PEs.append(PE)
13    return Es, KEs, PEs
14
15
16 def GetAverages(psis, xs_g, freqs_g):
17     """Return average position and momentum lists using FFT."""
18     dx = xs_g[1] - xs_g[0]
19     N = len(xs_g)
20     positions, momenta = [], []
21     for psi_g in psis:
22         pos = np.real(np.sum(np.abs(psi_g)**2 * xs_g) * dx)
23         positions.append(pos)
24         phi = np.fft.fft(psi_g)
25         mom = np.real(dx / N * np.sum(freqs_g * np.abs(phi)**2))
26         momenta.append(mom)
27     return positions, momenta
28
29
30 Es, KEs, PEs = GetEnergy(arrays, freqs, V, xs)
31 positions, momenta = GetAverages(arrays, xs, freqs)
32
33 fig, ax = plt.subplots(2, 3)
34 fig.set_figwidth(22)
35 fig.set_figheight(12)
36
37 t_axis = np.array(range(len(Es))) * T
38
39 ax[0, 0].plot(t_axis, Es, label='Total Energy')
40 ax[0, 0].plot(t_axis, PEs, label='Potential Energy')
41 ax[0, 0].plot(t_axis, KEs, label='Kinetic Energy')
42 ax[0, 0].legend()
43 ax[0, 0].set_xlabel('Time')
44 ax[0, 0].set_ylabel('Energy')
45 ax[0, 0].set_title('Energy Components vs Time')
46
47 ax[0, 1].plot(t_axis, positions)
48 ax[0, 1].set_xlabel('Time')
49 ax[0, 1].set_ylabel('<x>')
50 ax[0, 1].set_title('Average Position vs Time')
51
52 ax[0, 2].plot(t_axis, momenta)
53 ax[0, 2].set_xlabel('Time')

```

```

53 ax[0, 2].set_xlabel('Time')
54 ax[0, 2].set_ylabel('<p>')
55 ax[0, 2].set_title('Average Momentum vs Time')
56
57 ax[1, 0].plot(xs, np.abs(arrays[50])**2)
58 ax[1, 0].set_xlabel('x')
59 ax[1, 0].set_ylabel('Probability Density')
60 ax[1, 0].set_title('|\psi(x)|^2 at T = 50')
61
62 psi_k_plot = np.fft.fft(arrays[50])
63 ax[1, 1].plot(freqs, np.abs(psi_k_plot)**2, '.')
64 ax[1, 1].set_xlabel('k')
65 ax[1, 1].set_ylabel('|\psi(k)|^2')
66 ax[1, 1].set_title('Momentum Distribution at T = 50')
67
68 ax[1, 2].plot(xs, V)
69 ax[1, 2].set_xlabel('x')
70 ax[1, 2].set_ylabel('V(x)')
71 ax[1, 2].set_title('Potential Energy V(x)')
72
73 plt.tight_layout()
74 plt.show()
75
76
77

```



Total energy is conserved throughout the split-operator evolution; $\langle x \rangle$ oscillates within the well while the symmetric momentum distribution about $k = 0$ confirms the bound-state nature of this below-barrier superposition.

✓ Answer (end)

✓ c. Animate

We now want to animate our system. Because we're going to do this a couple times with different potential, let's go ahead and write an animate function `def animateMe(arrays, V)` that goes ahead and finds the `max_value` and does calls the animation.

Then go ahead and animate your wave-function.

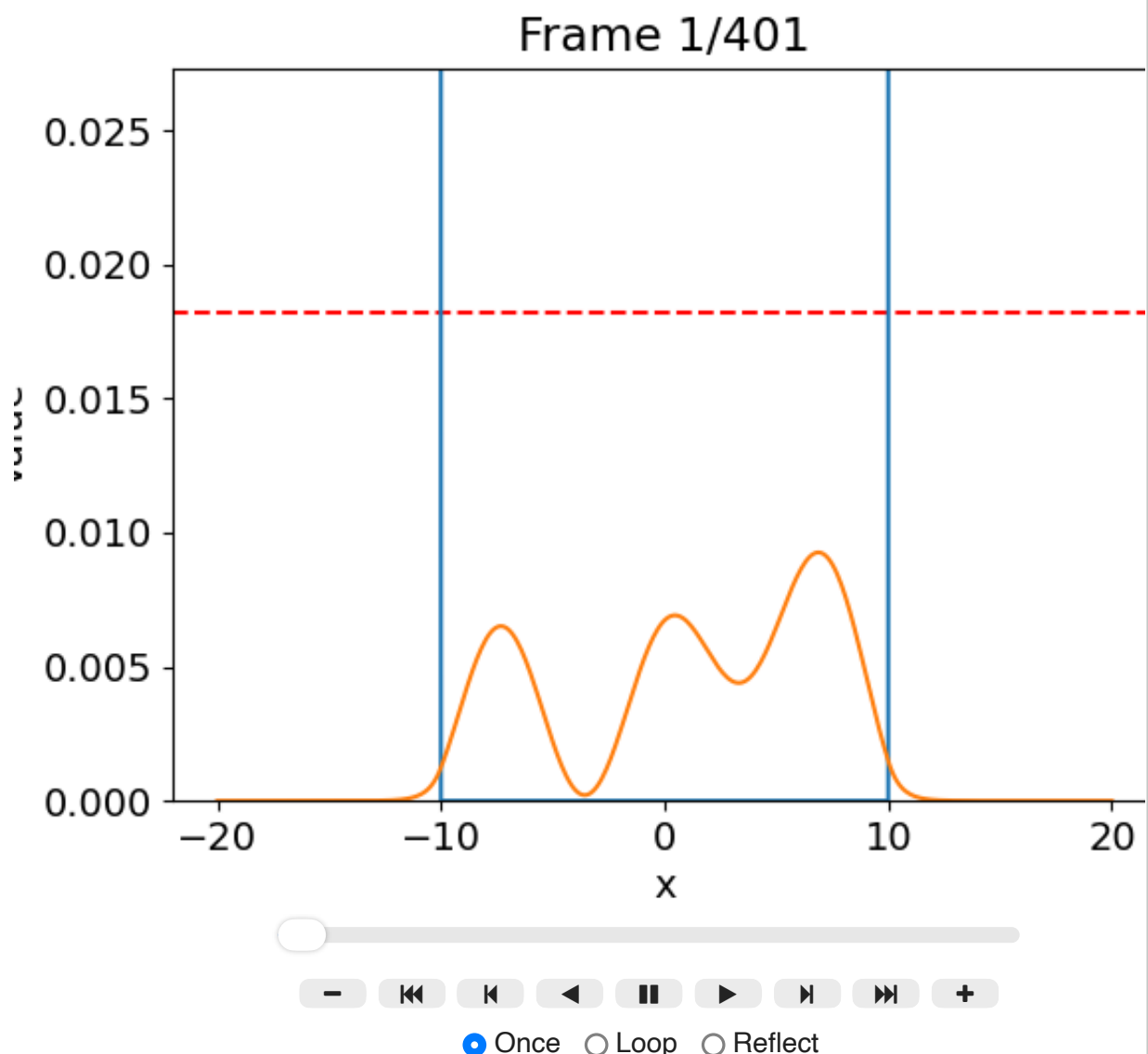
? Answer (start)

```
1 def animateMe(arrays_in, V_arr, xs_arr, freqs_arr):
2     global arrays
3     arrays = arrays_in
4     max_value = np.max([np.max(np.abs(arr)**2) for arr in arrays_in])
5     dx = xs_arr[1] - xs_arr[0]
6     N = len(xs_arr)
7     psi0_a = arrays[0]
8     PE0 = np.real(np.sum(np.abs(psi0_a)**2 * V_arr) * dx)
9     phi0 = np.fft.fft(psi0_a)
10    KE0 = np.real(dx / N * np.sum(freqs_arr**2 / 2.0 * np.abs(phi0)**2))
11    energy_val = PE0 + KE0
```

```

12     if abs(energy_val) < 1e-10:
13         energy_val = 1e-10
14
15     fig, ax_a = plt.subplots()
16     skip = 10
17     anim = FuncAnimation(fig, update, frames=len(arrays) // skip,
18                         fargs=(skip, xs_arr, None, V_arr, max_
19                             display(HTML(anim.to_jshtml())))
20     plt.close()
21
22
23 animateMe(arrays, V, xs, freqs)

```



✓ Answer (end)

✓ d. Time evolution of a "stationary" gaussian

We now would like to go ahead and do a time-evolution of a stationary Gaussian with the following parameters.

- $\psi_{\text{init}} = \frac{1}{N} \exp[-\frac{1}{2}x^2]$
- $L = 200$
- $\Delta x = 0.1$
- $\Delta t = 0.01$
- $T = 0.1$
- The potential is zero

Go ahead and write a `def SetupGauss()` function and produce the same plots and animations as above

? Answer (start)

```

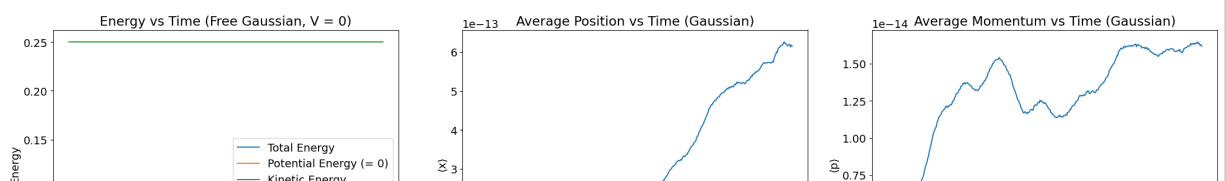
1 def SetupGauss():
2     L_g = 200
3     delta_x_g = 0.1
4     dt_g = 0.01
5     T_g = 0.1
6     xs_g = SetupGrid(L_g, delta_x_g)
7     freqs_g = np.fft.fftfreq(len(xs_g), delta_x_g) * (2 * np.pi)
8     V_g = np.zeros(len(xs_g))
9
10    psi_g = np.exp(-0.5 * xs_g**2).astype(complex)
11    psi_g /= np.sqrt(np.sum(np.abs(psi_g)**2) * delta_x_g)
12    return psi_g, V_g, dt_g, T_g, xs_g, freqs_g
13
14
15 psi_gauss, V_gauss, dt_gauss, T_gauss, xs_gauss, freqs_gauss =
16 arrays_gauss = [psi_gauss]
17 for step in range(400):
18     psi_gauss = TimeEvolution(T_gauss, dt_gauss, V_gauss, psi_g)
19     arrays_gauss.append(psi_gauss)
20
21 Es_g, KEs_g, PEs_g = GetEnergy(arrays_gauss, freqs_gauss, V_gau
22 positions_g, momenta_g = GetAverages(arrays_gauss, xs_gauss, fr
23
24 fig, ax = plt.subplots(2, 3)
25 fig.set_figwidth(22)
26 fig.set_figheight(12)

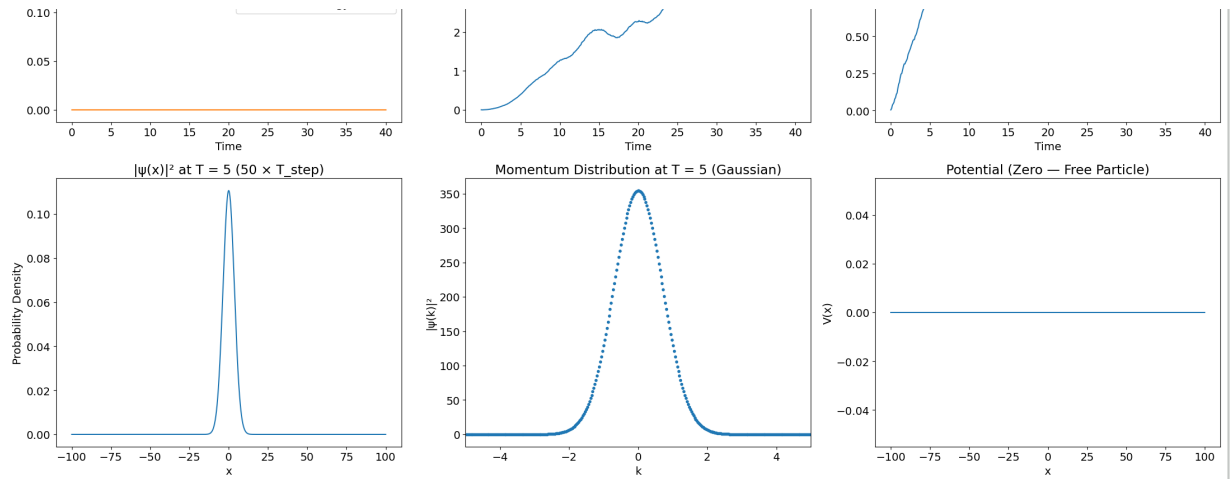
```

```

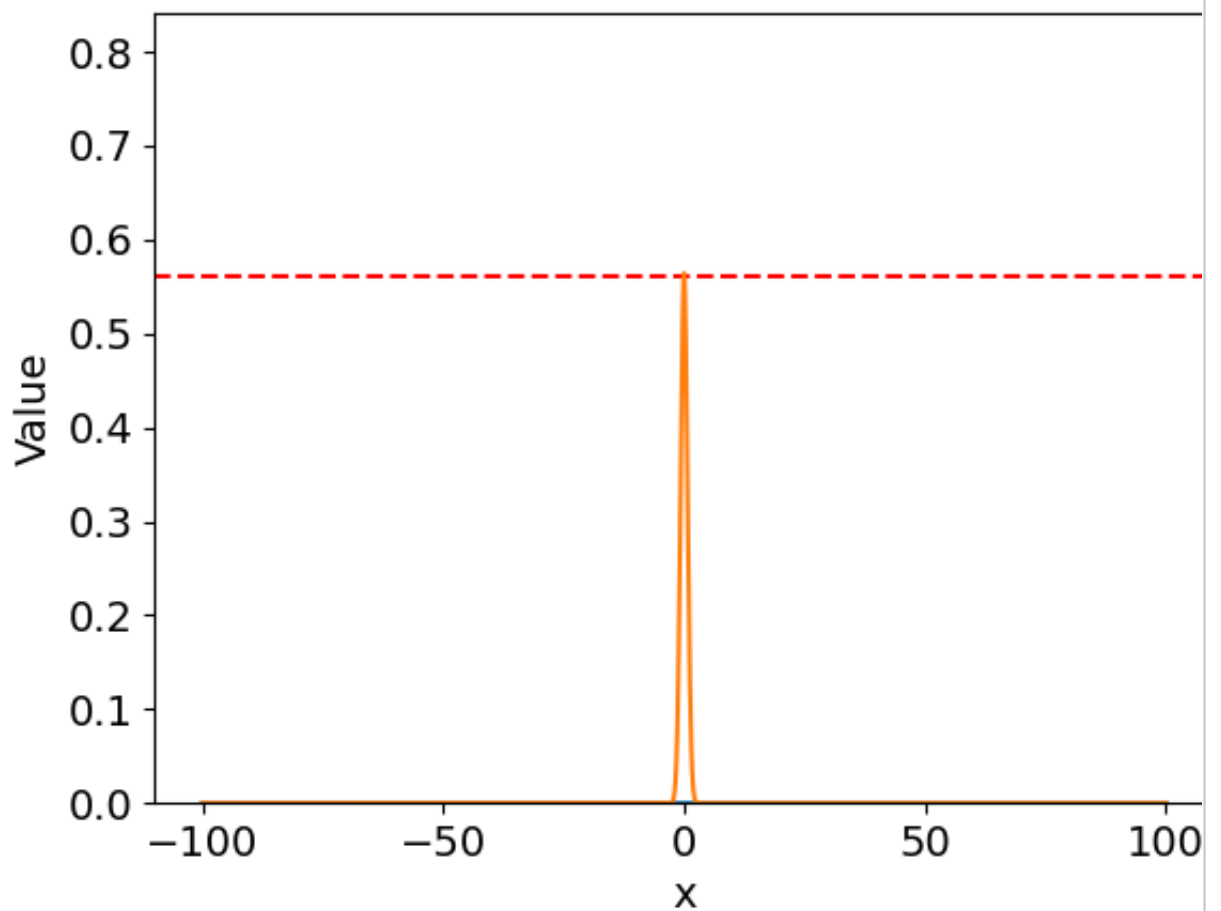
27
28 t_axis_g = np.array(range(len(Es_g))) * T_gauss
29
30 ax[0, 0].plot(t_axis_g, Es_g, label='Total Energy')
31 ax[0, 0].plot(t_axis_g, PEs_g, label='Potential Energy (= 0)')
32 ax[0, 0].plot(t_axis_g, KEs_g, label='Kinetic Energy')
33 ax[0, 0].legend()
34 ax[0, 0].set_xlabel('Time')
35 ax[0, 0].set_ylabel('Energy')
36 ax[0, 0].set_title('Energy vs Time (Free Gaussian, V = 0)')
37
38 ax[0, 1].plot(t_axis_g, positions_g)
39 ax[0, 1].set_xlabel('Time')
40 ax[0, 1].set_ylabel('<x>')
41 ax[0, 1].set_title('Average Position vs Time (Gaussian)')
42
43 ax[0, 2].plot(t_axis_g, momenta_g)
44 ax[0, 2].set_xlabel('Time')
45 ax[0, 2].set_ylabel('<p>')
46 ax[0, 2].set_title('Average Momentum vs Time (Gaussian)')
47
48 ax[1, 0].plot(xs_gauss, np.abs(arrays_gauss[50])**2)
49 ax[1, 0].set_xlabel('x')
50 ax[1, 0].set_ylabel('Probability Density')
51 ax[1, 0].set_title('|ψ(x)|² at T = 5 (50 × T_step)')
52
53 psi_k_gauss = np.fft.fft(arrays_gauss[50])
54 ax[1, 1].plot(freqs_gauss, np.abs(psi_k_gauss)**2, '.')
55 ax[1, 1].set_xlabel('k')
56 ax[1, 1].set_ylabel('|ψ(k)|²')
57 ax[1, 1].set_title('Momentum Distribution at T = 5 (Gaussian)')
58 ax[1, 1].set_xlim(-5, 5)
59
60 ax[1, 2].plot(xs_gauss, V_gauss)
61 ax[1, 2].set_xlabel('x')
62 ax[1, 2].set_ylabel('V(x)')
63 ax[1, 2].set_title('Potential (Zero – Free Particle)')
64
65 plt.tight_layout()
66 plt.show()
67
68
69 animateMe(arrays_gauss, V_gauss, xs_gauss, freqs_gauss)

```





Frame 1/401



☒ Once ☐ Loop ☐ Reflect

A zero-momentum Gaussian ($\langle p \rangle = \langle x \rangle = 0$) spreads symmetrically over time due to quantum dispersion while conserving total (purely kinetic) energy, since there is no restoring potential to confine it.

✓ Answer (end)

✓ e. Hitting a Potential Barrier

Let us now consider a potential barrier with

- $V(x) = V_0$ for $-a < x < a$ and 0 otherwise where $V_0 = 10$ and $a = 0.5$
- $L = 60$
- $\Psi_{init} = \frac{1}{N} \exp[-0.5(x + 20)^2 + 5ix]$
- $T = 0.05$
- $dt = 0.1$

In addition to the standard plots, please also make a plot of the amount of the probability to the left and right of $x = 0$.

? Answer (start)

```
1 def SetupBarrier():
2     V0 = 10.0
3     a = 0.5
4     L_bar = 60
5     delta_x_bar = 0.1
6     dt_bar = 0.001 # fine time step for accuracy (problem dt=
7     T_bar = 0.05   # time per snapshot
```

```

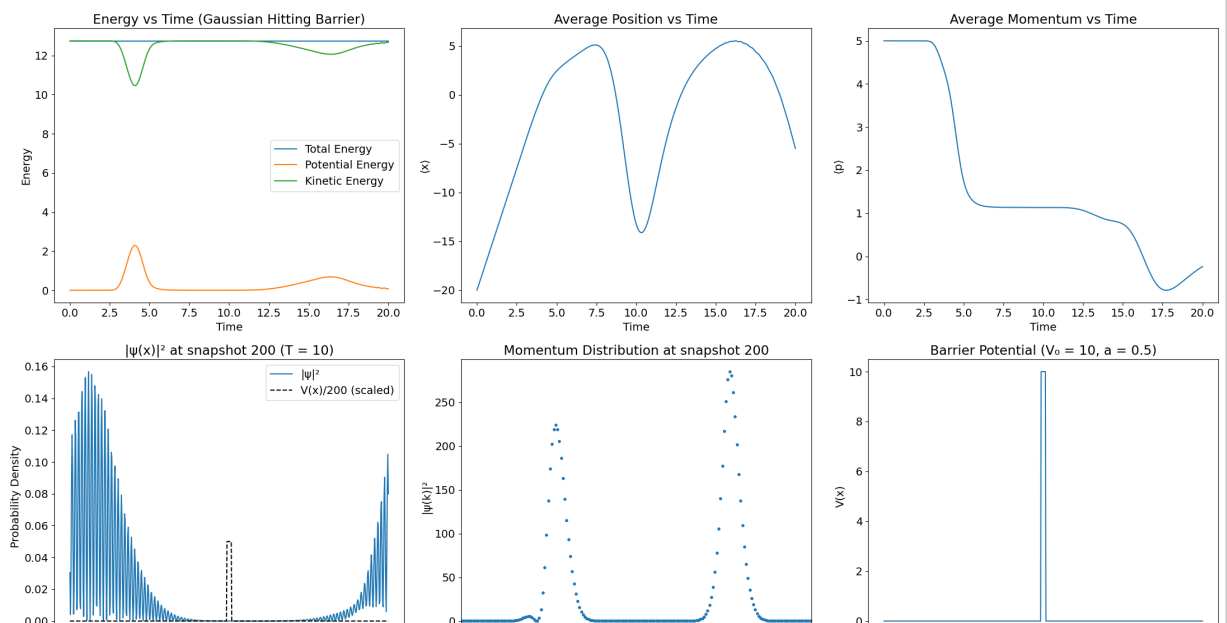
8     xs_bar = SetupGrid(L_bar, delta_x_bar)
9     freqs_bar = np.fft.fftfreq(len(xs_bar), delta_x_bar) * (2 *
10    V_bar = np.where(np.abs(xs_bar) < a, V0, 0.0)
11
12    # Gaussian centered at x = -20 with momentum kick k = 5
13    psi_bar = np.exp(-0.5 * (xs_bar + 20)**2 + 5.j * xs_bar)
14    psi_bar /= np.sqrt(np.sum(np.abs(psi_bar)**2) * delta_x_bar)
15    return psi_bar, V_bar, dt_bar, T_bar, xs_bar, freqs_bar
16
17
18 psi_b, V_b, dt_b, T_b, xs_b, freqs_b = SetupBarrier()
19 arrays_b = [psi_b]
20 for step in range(400):
21     psi_b = TimeEvolution(T_b, dt_b, V_b, psi_b, freqs_b)
22     arrays_b.append(psi_b)
23
24 Es_b, KEs_b, PEs_b = GetEnergy(arrays_b, freqs_b, V_b, xs_b)
25 positions_b, momenta_b = GetAverages(arrays_b, xs_b, freqs_b)
26
27 fig, ax = plt.subplots(2, 3)
28 fig.set_figwidth(22)
29 fig.set_figheight(12)
30 t_axis_b = np.array(range(len(Es_b))) * T_b
31
32 ax[0, 0].plot(t_axis_b, Es_b, label='Total Energy')
33 ax[0, 0].plot(t_axis_b, PEs_b, label='Potential Energy')
34 ax[0, 0].plot(t_axis_b, KEs_b, label='Kinetic Energy')
35 ax[0, 0].legend()
36 ax[0, 0].set_xlabel('Time')
37 ax[0, 0].set_ylabel('Energy')
38 ax[0, 0].set_title('Energy vs Time (Gaussian Hitting Barrier)')
39
40 ax[0, 1].plot(t_axis_b, positions_b)
41 ax[0, 1].set_xlabel('Time')
42 ax[0, 1].set_ylabel('<x>')
43 ax[0, 1].set_title('Average Position vs Time')
44
45 ax[0, 2].plot(t_axis_b, momenta_b)
46 ax[0, 2].set_xlabel('Time')
47 ax[0, 2].set_ylabel('<p>')
48 ax[0, 2].set_title('Average Momentum vs Time')
49
50 ax[1, 0].plot(xs_b, np.abs(arrays_b[200])**2, label='|ψ|²')
51 ax[1, 0].plot(xs_b, V_b / 200, 'k--', label='V(x)/200 (scaled)')
52 ax[1, 0].set_xlabel('x')
53 ax[1, 0].set_ylabel('Probability Density')
54 ax[1, 0].set_title('|ψ(x)|² at snapshot 200 (T = 10)')
55 ax[1, 0].legend()

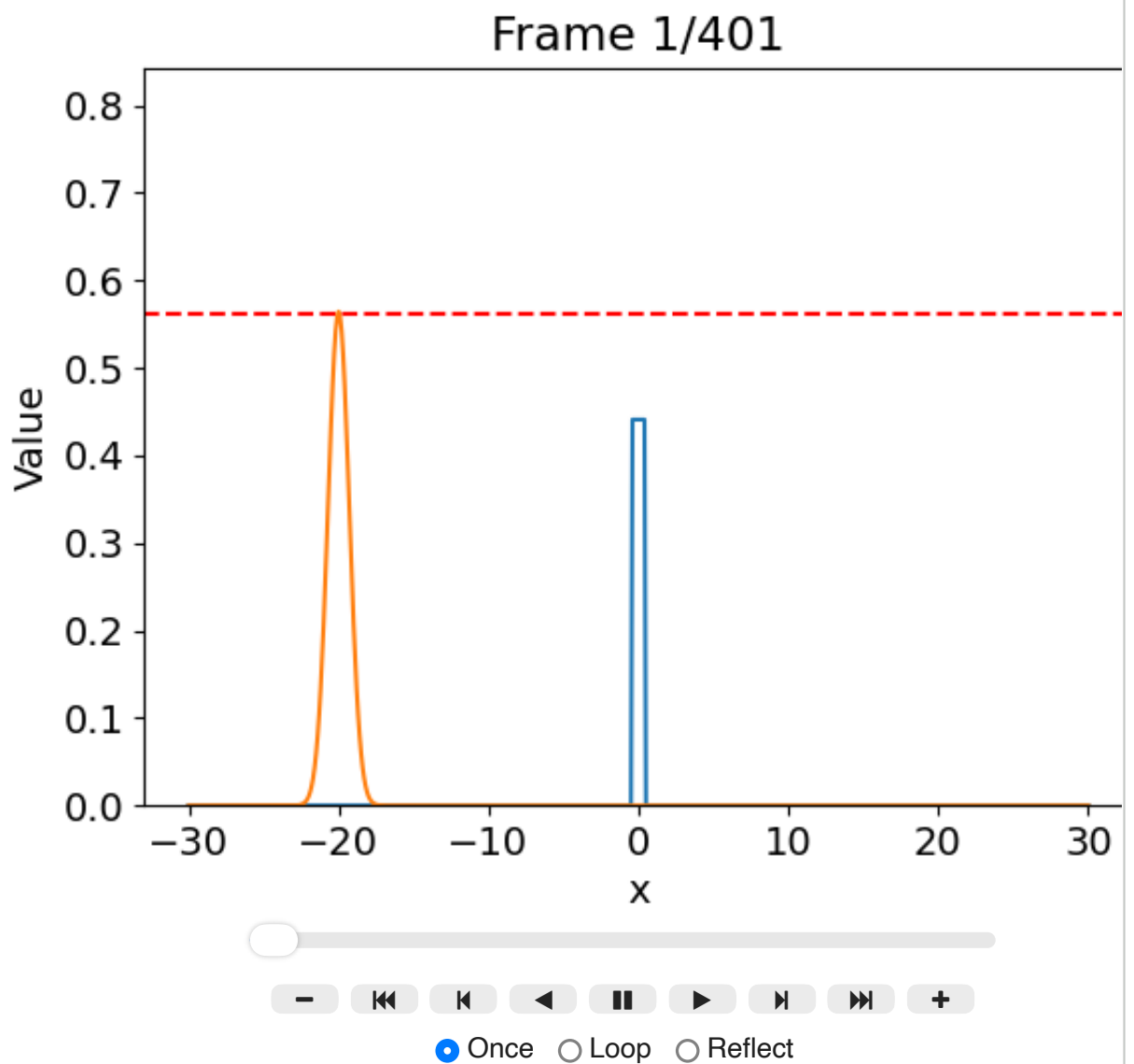
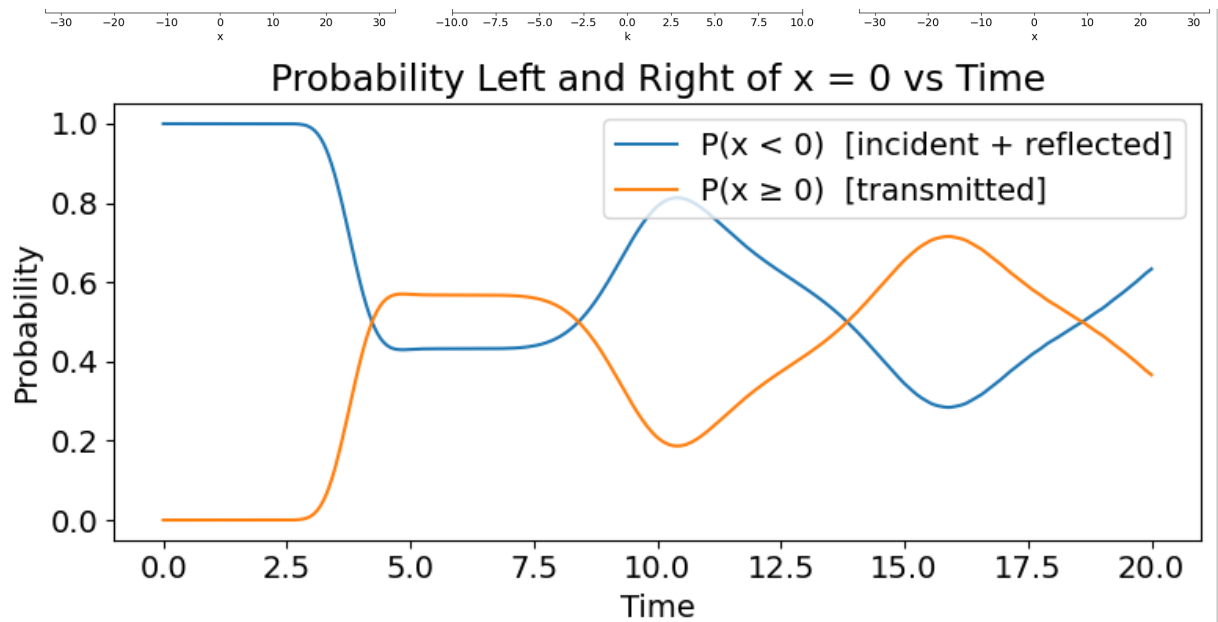
```

```

56
57 psi_k_b = np.fft.fft(arrays_b[200])
58 ax[1, 1].plot(freqs_b, np.abs(psi_k_b)**2, '.')
59 ax[1, 1].set_xlabel('k')
60 ax[1, 1].set_ylabel('|\psi(k)|^2')
61 ax[1, 1].set_title('Momentum Distribution at snapshot 200')
62 ax[1, 1].set_xlim(-10, 10)
63
64 ax[1, 2].plot(xs_b, V_b)
65 ax[1, 2].set_xlabel('x')
66 ax[1, 2].set_ylabel('V(x)')
67 ax[1, 2].set_title('Barrier Potential (V_0 = 10, a = 0.5)')
68
69 plt.tight_layout()
70 plt.show()
71
72 # Probability to the left and right of x = 0
73 dx_b = xs_b[1] - xs_b[0]
74 prob_left = [np.sum(np.abs(arr[xs_b < 0])**2) * dx_b for arr
75 prob_right = [np.sum(np.abs(arr[xs_b >= 0])**2) * dx_b for arr
76
77 plt.figure(figsize=(8, 4))
78 plt.plot(t_axis_b, prob_left, label='P(x < 0) [incident + ref
79 plt.plot(t_axis_b, prob_right, label='P(x ≥ 0) [transmitted]')
80 plt.xlabel('Time')
81 plt.ylabel('Probability')
82 plt.title('Probability Left and Right of x = 0 vs Time')
83 plt.legend()
84 plt.tight_layout()
85 plt.show()
86
87 animateMe(arrays_b, V_b, xs_b, freqs_b)

```





Since the incident kinetic energy ($KE = k^2/2 = 12.5$) exceeds the barrier ($V_0 = 10$), most probability transmits through, but quantum mechanics still predicts partial reflection—visible as residual probability remaining on the left side after the collision.

✓ Answer (end)

Acknowledgements:

- Bryan Clark (original)

© Copyright 2025

Double-click (or enter) to edit